

Answer with Explanation

Q01. When we run this code, the computer will show two lines of text. The first line says **"Hello 1"** and the second line says **"Hello 2"** The word **"Hello"** is just a friendly greeting, like saying hi to someone.

So, when you run this code, you'll see the computer saying **"Hello 1"** and then **"Hello 2"** on the screen. It's a simple way for a computer to communicate with us and show us messages.

Q02. The first line says **"print('Ok 1')"**. When you run this line, the computer will display the text **"Ok 1"** on the screen.

The second line says **"print('Ok 2')"**. When you run this line, the computer will display the text **"Ok 2"** on the screen.

The third line says **"print('Ok 3')"**. When you run this line, the computer will display the text **"Ok 3"** on the screen.

So, when you run this code, you will see three lines of text: **"Ok 1"**, **"Ok 2"**, and **"Ok 3"**. Each line will be displayed one after the other, so it will look like this:

Ok 1

Ok 2

Ok 3

It's a way for the computer to show different messages or information to us in a sequence.

Q03. In the first line, the code says **"print("Hello" + " " + "Ahmed")"**. The plus sign (+) here is used to combine or concatenate strings together. So, when you run this line, the computer will display the text **"Hello Ahmed"** on the screen. The plus sign joins the word **"Hello"**, a space (" "), and the name **"Ahmed"** together to form a single string.

The second line says `"print('Hello' + ' ' + 'Adel')"`. Here, we have the same concatenation using the plus sign. When you run this line, the computer will display **"Hello Adel"** on the screen. Again, the plus sign combines the word **"Hello"**, a space (`' '`), and the name **"Adel"** to create the final string.

The third line says `"print('Hello' + ' ' + 'Amr')"`. Similar to the previous lines, this line will display **"Hello Amr"** on the screen. The plus sign combines the word **"Hello"**, a space (`' '`), and the name **"Amr"** to create the string.

The fourth line says `"print('My' + ' ' + 'name is ' + '"' + "Ali" + '"' )"`. This line is a little more complex. When you run it, the computer will display **"My name is "Ali"** on the screen. Let's break it down:

- The plus sign is used to concatenate the strings together.
- The single quotes (`' '`) and double quotes (`" "`) are used to specify the beginning and end of a string.
- The double quotes within the string are used to include the name **"Ali"** with quotation marks around it.

So, in this line, the word **"My"**, a space (`" "`), the phrase **"name is "**, the double quotes (`'"'`), the name **"Ali"**, and the closing double quotes (`'"'`) are combined to create the final string.

Q04. In the first line, the code says `"print(7+3)"`. The plus sign (+) in this context is used for Addition. When you run this line, the computer will calculate the sum of **7** and **3**, which is **10**. So, it will display the result, **10**, on the screen.

In the second line, the code says `"print(7-3)"`. The minus sign (-) in this context is used for subtraction. When you run this line, the computer will calculate the difference between **7** and **3**, which is **4**. So, it will display the result, **4**, on the screen.

In the third line, the code says `"print(73)"`. The asterisk (*) in

this context is used for multiplication. When you run this line, the computer will calculate the product of **7** and **3**, which is **21**. So, it will display the result, **21**, on the screen.

In the fourth line, the code says "**print(7/3)**". The forward slash (/) in this context is used for division. When you run this line, the computer will calculate the division of **7** by **3**, which is approximately **2.33333** (repeating). So, it will display the result, **2.33333**, on the screen.

It's important to note that the result of the division operation is a **floating-point** number (a decimal) because the division of two numbers may not always result in a whole number.

In summary, these lines of code demonstrate basic arithmetic operations in Python. They show how to add, subtract, multiply, and divide numbers using the appropriate mathematical operators.

Q05. In the first line, the code says "**print(7//3)**". The double forward slash (//) in this context is used for integer division or floor division. When you run this line, the computer will calculate the division of **7** by **3**, but instead of returning a floating-point number, **it will return the largest whole number that is less than or equal to the division result**. In this case, **7** divided by **3** is approximately **2.33333**, but the integer division will return the whole number **2**. So, it will display the result, **2**, on the screen.

In the second line, the code says "**print(5**3)**". The double asterisk (**) in this context is used for exponentiation or raising a number to a power. When you run this line, it will calculate **5 raised to the power of 3**, which means **multiplying 5 by itself three times**. So, it will perform the calculation **5 * 5 * 5**, which equals **125**. Therefore, it will display the result, **125**, on the screen.

In summary, the first line demonstrates integer division, which returns the whole number part of the division result. The second line demonstrates exponentiation, which allows you to raise a number to a specific power.

Q06. In the first line, the code assigns the value **7** to a variable named **"num1"**. Variables are like containers that hold values, and in this case, **"num1"** is assigned the value **7**.

In the second line, the code assigns the value **3** to a variable named **"num2"**. Similarly, **"num2"** is assigned the value **3**.

In the third line, the code creates a new variable called **"result"** and assigns it the value of the sum of **"num1"** and **"num2"**. Since **"num1"** is **7** and **"num2"** is **3**, the sum of **7** and **3** is **10**. Therefore, the variable **"result"** will have the value **10**.

Finally, in the fourth line, the code uses the **"print"** command to display the value of the variable **"result"** on the screen. When you run this code, it will print the value **10** on the screen.

So, the output of this code will be:

10

The code demonstrates how variables can store values, and we can perform operations on those variables to get the desired result.

Q07. In the first line, the code assigns the string **'Ahmed'** to a variable called **"name"**. Variables are like containers that can hold different types of data, such as numbers, text, or in this case, a name.

In the second line, the code creates a new variable called **"say_hello"** and assigns it a value by combining the string **'Hello '** with the value stored in the **"name"** variable. The plus sign (+) is used to concatenate or join strings together. In this case, it concatenates the string **'Hello '** with the value stored in the **"name"** variable, which is **'Ahmed'**. So, the **"say_hello"** variable will have the value **'Hello Ahmed'**.

Finally, in the third line, the code uses the **"print"** command to display the value of the **"say_hello"** variable on the screen. When you

run this code, it will print the string **'Hello Ahmed'** on the screen.

So, the output of this code will be:

Hello Ahmed

The code demonstrates how you can use variables to store values and combine strings together to create more meaningful messages or sentences.

Q08. In the first line, the code assigns the value **'Ahmed'** to the variable **"name1"**, **'Adel'** to the variable **"name2"**, and **'Amr'** to the variable **"name3"**. The values are assigned to the variables in the same order they appear: **"Ahmed"** to **"name1"**, **"Adel"** to **"name2"**, and **"Amr"** to **"name3"**.

In the second line, the code uses the **"print"** command to display the concatenation of the variables **"name1"**, **"name2"**, and **"name3"**. The plus sign (+) is used to concatenate or join strings together. In this case, it concatenates the values stored in the variables with a **space** (' ') in between each one. So, it will display the string **'Ahmed Adel Amr'** on the screen.

Therefore, when you run this code, it will print the string **'Ahmed Adel Amr'** on the screen.

The code demonstrates how you can assign values to multiple variables in a single line and then concatenate those variables to create a combined string for printing or further manipulation.

Q09. In the first line, the code assigns the value **1** to the variable **"num"**, the string **'Ali'** to the variable **"name"**, the value **4500.66** to the variable **"salary"**, and the value **True** to the variable **"is_active"**. The values are assigned to the variables in the same order they appear: **1** to **"num"**, **'Ali'** to **"name"**, **4500.66** to **"salary"**, and **True** to **"is_active"**.

In the second line, the code uses the **"print"** command to display the value stored in the **"num"** variable. When you run this code, it will

print the value **1** on the screen.

In the third line, the code uses the **"print"** command to display the value stored in the **"name"** variable. When you run this code, it will print the string **'Ali'** on the screen.

In the fourth line, the code uses the **"print"** command to display the value stored in the **"salary"** variable. When you run this code, it will print the value **4500.66** on the screen.

In the fifth line, the code uses the **"print"** command to display the value stored in the **"is_active"** variable. When you run this code, it will print the value **True** on the screen.

So, the output of this code will be:

1

Ali

4500.66

True

The code demonstrates how you can assign values to multiple variables in a single line and then access and print those variables individually.

Q10. In the first line, the code assigns the value **'Adel'** to the variable **"name"**.

The second line uses the **"print"** command to display the value of the **"name"** variable, which is **'Adel'**. When you run this code, it will print **'Adel'** on the screen.

In the third line, the code assigns the value **'Ahmed'** to the variable **"name"**. It **overwrites** the previous value of **'Adel'**.

The fourth line assigns the value **'Amr'** to the variable **"name"**. It also **overwrites** the previous value of **'Ahmed'**.

Finally, the fifth line uses the **"print"** command to display the value

of the **"name"** variable, which is now **'Amr'**. When you run this code, it will print **'Amr'** on the screen.

So, the output of this code will be:

Adel

Amr

The code demonstrates how you can assign and reassign values to a variable. The variable will hold the most recent value assigned to it, and any previous value will be overwritten.

Q11. In the first line, the code assigns the integer value **733** to the variable **"var1"**.

In the second line, the code assigns the float value **99.55** to the variable **"var2"**. Floats are used to represent decimal numbers.

In the third line, the code assigns the boolean value **True** to the variable **"var3"**. Booleans can have two possible values: **True** or **False**, representing logical values.

In the fourth line, the code assigns the string value **'Hello'** to the variable **"var4"**. Strings are used to represent text or characters.

In the fifth line, the code assigns the string value **"Hi"** to the variable **"var5"**. Strings can be enclosed in either **single quotes** (**'**) or **double quotes** (**"**).

In the sixth line, the code uses the **"type"** function to determine and print the data type of the variable **"var1"**. The **"type"** function returns the data type of a given value. In this case, it will print **"<class 'int'>"**, indicating that **"var1"** is of type **integer**.

In the seventh line, the code uses the **"type"** function to determine and print the data type of the variable **"var2"**. It will print **"<class 'float'>"**, indicating that **"var2"** is of type **float**.

In the eighth line, the code uses the **"type"** function to determine

and print the data type of the variable `"var3"`. It will print `<class 'bool'>`, indicating that `"var3"` is of type `boolean`.

In the ninth line, the code uses the `"type"` function to determine and print the data type of the variable `"var4"`. It will print `<class 'str'>`, indicating that `"var4"` is of type `string`.

In the tenth line, the code uses the `"type"` function to determine and print the data type of the variable `"var5"`. It will print `<class 'str'>`, indicating that `"var5"` is also of type `string`.

So, the output of this code will be:

```
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'str'>
<class 'str'>
```

The code demonstrates how you can use the `"type"` function to determine the data type of a variable or value, which can be useful for understanding and manipulating different types of data.

Q12. The code you provided assigns a **multi-line string** to the variable `"my_str"` and then prints its value. Here's the explanation:

In the first line, the code assigns a **multi-line string** to the variable `"my_str"`. The string is enclosed in **triple quotes** (`"""`) to indicate that it spans multiple lines. The string contains the famous nursery rhyme `"Twinkle Twinkle Little Star"` and two additional lines.

In the second line, the code uses the `"print"` command to display the value of the variable `"my_str"` on the screen. When you run this code, it will print the entire **multi-line string** on the screen, preserving the line breaks and indentation.

So, the output of this code will be:

```
Twinkle Twinkle Little Star
How I wonder what you are
```



```
Up above the sky so high  
Like a diamond in the sky
```

The code demonstrates how you can assign and print multi-line strings, which is useful for storing and displaying longer blocks of text or structured content.

Q13. The code defines a variable named `'my_str'` and assigns a multi-line string to it. The string is enclosed in **triple single quotes** (`'''`), which allows it to span multiple lines.

2. The content of the `'my_str'` variable is as follows:

```
Welcome to Hassouna Academy  
I love Python  
Now Create Account On www.hassouna-academy.com
```

3. The code then proceeds to print the value of the `'my_str'` variable using the `'print()'` function.

4. When you run the code, it will output the following:

```
Welcome to Hassouna Academy  
I love Python  
Now Create Account On www.hassouna-academy.com
```

Each line of the multi-line string is printed exactly as it is, and the output will be displayed on the console.

If you have any more code or questions you'd like me to explain, feel free to share!

Q14. In the first line, the code prints the string `'A '` repeated 5 times. The asterisk (`*`) operator in this context is used for string repetition. When you run this line, it will print `'A A A A A '` on the screen. The string `'A '` is repeated 5 times, resulting in a sequence of five `'A'` characters followed by a **space**.

In the second line, the code assigns the string ' **AMR** ' to the variable "name". The string is assigned with **leading and trailing spaces**.

In the third line, the code prints the value of the variable "name" repeated **3** times. Similar to the first line, it uses the asterisk (*) operator for string repetition. When you run this line, it will print ' **AMR AMR AMR** ' on the screen. The string ' **AMR** ' is repeated 3 times, resulting in **three copies** of the string with **leading and trailing spaces**.

So, the output of this code will be:

```
A A A A A
AMR AMR AMR
```

The code demonstrates how you can repeat strings multiple times using the asterisk (*) operator, which can be useful for generating repetitive patterns or producing formatted output.

Q15. In the first line, the code assigns the escape sequence '\n' to the variable "str1". '\n' represents a newline character.

In the second line, the code assigns the Unicode character named "**copyright sign**" to the variable "str2" using the escape sequence '\N{**copyright sign**'. This will store the copyright sign symbol (©) in the variable.

In the third line, the code assigns the Unicode character named "**registered sign**" to the variable "str3" using the escape sequence '\N{**registered sign**'. This will store the registered sign symbol (®) in the variable.

In the fourth line, the code assigns the Unicode character named "**up-down arrow**" to the variable "str4" using the escape sequence '\N{**up down arrow**'. This will store the up-down arrow symbol (↕) in the variable.

In the fifth line, the code assigns the Unicode character named

"**left-right arrow**" to the variable "**str5**" using the escape sequence '`\N{left right arrow}`'. This will store the left-right arrow symbol (**↔**) in the variable.

In the sixth line, the code assigns the hexadecimal value '`\x41`' to the variable "**str6**". This will store the character '**A**' in the variable. In Unicode, '**A**' has a hexadecimal value of **41**.

In the seventh line, the code assigns the Unicode value '`\u0042`' to the variable "**str7**". This will store the character '**B**' in the variable. In Unicode, '**B**' has a hexadecimal value of **42**.

In the eighth line, the code assigns the Unicode value '`\U00000043`' to the variable "**str8**". This will store the character '**C**' in the variable. In Unicode, '**C**' has a hexadecimal value of **43**.

In the ninth line, the code uses the "**print**" command to display the string '**This For New Line**', followed by the value of "**str1**", followed by the string '**OK**'. When you run this line, it will print '**This For New Line**', followed by a new line, followed by '**OK**'.

In the tenth line, the code uses the "**print**" command to display the values of "**str2**", "**str3**", "**str4**", and "**str5**". When you run this line, it will print the corresponding Unicode symbols: '**© ® ↔ ↔**'.

In the eleventh line, the code uses the "**print**" command to display the values of "**str6**", "**str7**", and "**str8**". When you run this line, it will print the characters: '**A B C**'.

So, the output of this code will be:

This For New Line

OK

© ® ↔ ↔

A B C

The code demonstrates different ways to represent special characters and Unicode symbols in Python strings using escape sequences and Unicode code points.

Q16. In the first line, the code assigns the string `'Hello \'Ahmed\''` to the variable `"str1"`. The **backslash** (`\`) is an escape character used to indicate that the **single quote** (`'`), which follows the **backslash**, is a part of the string and not the **closing quote**. This allows the **single quote** to be included within the string.

In the second line, the code assigns the string `'Hello "Adel"'` to the variable `"str2"`. The **double quotes** (`"`) are used to enclose the string, and the **backslashes** (`\`) before the **double quotes** are **escape characters** that indicate the **double quotes** are part of the string and not the **closing quotes**.

In the third line, the code assigns the string `'Hello \\Amr\\'` to the variable `"str3"`. The **backslashes** (`\`) in this case are **escape characters** that indicate that the **backslashes** should be included as **literal characters** in the string.

In the fourth line, the code assigns the string `'\fFormfeed is\n{FF}'` to the variable `"str4"`. The `\f` represents a form feed character, and `\n{FF}` represents the Unicode character named **"Form Feed"**. When you run this line, it will print the **form feed character** followed by **"Formfeed is"** on the screen.

In the fifth line, the code assigns the string `'\nLinefeed is\n{LF}'` to the variable `"str5"`. The `\n` represents a line feed character, and `\n{LF}` represents the Unicode character named **"Line Feed"**. When you run this line, it will print the line feed character followed by **"Linefeed is"** on the screen.

In the sixth line, the code assigns the string `'Welcome to Egypt\rCarriage Return or \n{BEL}'` to the variable `"str6"`. The `\r` represents a carriage return character, and `\n{BEL}` represents the Unicode character named **"Bell"**. When you run this line, it will print **"Carriage Return or "** followed by the carriage return character and then **"Bell"** on the screen.

In the seventh line, the code attempts to print `"str7"`, but there is no variable assigned to `"str7"` in the given code. So, there will be an error.

In the eighth line, the code assigns the string `'Rad Cat\br\b\b\b\b\N{BS}'` to the variable `"str8"`. The `\b` represents a **backspace character**, and `\N{BS}` represents the Unicode character named **"Backspace"**. When you run this line, it will print **"Rad Cat"** followed by several **backspace characters** that erase the previously printed characters, and then **"Backspace"** on the screen.

In the ninth line, the code assigns the string `'Hello\t\N{TAB}World'` to the variable `"str9"`. The `\t` represents a tab character, and `\N{TAB}` represents the Unicode character named **"Horizontal Tab"**. When you run this line, it will print **"Hello"**, followed by a **tab character**, and then **"World"** on the screen.

In the tenth line, the code assigns the string `'Vertical\v\N{VT}Tab'` to the variable `"str10"`. The `\v` represents a vertical tab character, and `\N{VT}` represents the Unicode character named **"Vertical Tab"**. When you run this line, it will print **"Vertical"**, followed by a **vertical tab character**, and then **"Tab"** on the screen.

In the eleventh line, the code assigns the string `'Three Digits Octal:\101'` to the variable `"str11"`. The `\101` represents an **octal value (101)** that is converted to the corresponding character. In this case, it represents the character **'A'** in ASCII. When you run this line, it will print **"Three Digits Octal:A"** on the screen.

So, the output of this code will be:

Hello 'Ahmed'

Hello "Adel"

Hello \Amr

Formfeed is

Linefeed is

Welcome to Egypt

Carriage Return or

Rad CatBackspace

Hello World

Vertical

Tab

Three Digits Octal:A

The code demonstrates different escape sequences and Unicode characters that can be used in Python strings to represent special characters and control characters.

Q17. In the first line, the code creates a tuple named **"p1"** that contains three elements: the **integer 1**, the string **'Ahmed'**, and the **float 3900.50**.

In the second line, the code creates a tuple named **"p2"** that contains three elements: the **integer 2**, the string **'Adel'**, and the **float 4600.60**.

In the third line, the code creates a tuple named **"p3"** that contains three elements: the **integer 3**, the string **'Amr'**, and the **float 4500.55**.

In the fourth line, the code uses the **"print"** command to display the value of the tuple **"p1"**. When you run this line, it will print the entire tuple **"p1"** on the screen: **(1, 'Ahmed', 3900.50)**.

In the fifth line, the code uses the **"print"** command to display the value of the tuple **"p2"**. When you run this line, it will print the entire tuple **"p2"** on the screen: **(2, 'Adel', 4600.60)**.

In the sixth line, the code uses the **"print"** command to display the value of the tuple **"p3"**. When you run this line, it will print the entire tuple **"p3"** on the screen: **(3, 'Amr', 4500.55)**.

In the seventh line, the code uses the **"print"** command to display individual elements of the tuple **"p1"**. It prints the first element of **"p1"** (the **integer 1**), followed by a **space**, then the second element (the **string 'Ahmed'**), followed by another **space**, and finally the third element (the **float 3900.50**). When you run this line, it will print: **1 Ahmed 3900.5**.

In the eighth line, the code uses the **"type"** function to determine

and print the data type of the tuple **"p1"**. It will print **"<class 'tuple'>"**, indicating that **"p1"** is of type tuple.

In the ninth line, the code uses the **"type"** function to determine and print the data type of each element in the tuple **"p1"**. It will print **"<class 'int'>"**, **"<class 'str'>"**, and **"<class 'float'>"**, indicating that the first element is of type **integer**, the second element is of type **string**, and the third element is of type **float**.

So, the output of this code will be:

```
(1, 'Ahmed', 3900.50)
(2, 'Adel', 4600.60)
(3, 'Amr', 4500.55)
1 Ahmed 3900.5
<class 'tuple'>
<class 'int'> <class 'str'> <class 'float'>
```

The code demonstrates how to create tuples, access individual elements of a tuple, and determine the data type of tuples and their elements. Tuples are similar to lists, but they are immutable, meaning their elements cannot be changed once created.

Q18. In the first line, the code creates a list named **"numbers"** that contains three elements: the **integers 11, 22, and 33**.

In the second line, the code creates a list named **"names"** that contains three elements: the **strings 'Amr', 'Ali', and 'Ezz'**.

In the third line, the code uses the **"print"** command to display the value of the list **"numbers"**. When you run this line, it will print the entire list **"numbers"** on the screen: **[11, 22, 33]**.

In the fourth line, the code uses the **"print"** command to display the value of the list **"names"**. When you run this line, it will print the entire list **"names"** on the screen: **['Amr', 'Ali', 'Ezz']**.

In the fifth line, the code uses the **"print"** command to display the first element of the list **"numbers"**. List indexing starts from **0**, so

the first element is accessed with the **index 0**. When you run this line, it will print the value **11**.

In the sixth line, the code uses the **"print"** command to display the second element of the list **"numbers"**. The second element is accessed with the **index 1**. When you run this line, it will print the value **22**.

In the seventh line, the code uses the **"print"** command to display the third element of the list **"numbers"**. The third element is accessed with the **index 2**. When you run this line, it will print the value **33**.

In the eighth line, the code assigns the value **'Ak1'** to the first element of the list **"names"** by using list indexing. Since lists are mutable, you can modify individual elements by assigning new values to them.

In the ninth line, the code uses the **"print"** command to display the modified list **"names"**. When you run this line, it will print the updated list: **['Ak1', 'Ali', 'Ezz']**.

So, the output of this code will be:

```
[11, 22, 33]
['Amr', 'Ali', 'Ezz']
11
22
33
['Ak1', 'Ali', 'Ezz']
```

The code demonstrates how to create lists, access individual elements of a list using indexing, and modify list elements by assigning new values. Lists are mutable, meaning their elements can be changed after creation.

Q19. In the first line, the code creates a list named **"names"** that contains three elements: the strings **'Amr', 'Ali', and 'Ezz'**.

In the second line, the code uses the **"print"** command to display the value of the list **"names"**. When you run this line, it will print the

entire list **"names"** on the screen: **['Amr', 'Ali', 'Ezz']**.

In the third line, the code uses the **"append"** method to add the string **'Omar'** as a new element at the end of the list **"names"**. The **"append"** method modifies the list in place by adding the specified element. When you run this line, it will add **'Omar'** to the list.

In the fourth line, the code uses the **"print"** command to display the updated list **"names"**. When you run this line, it will print the modified list on the screen, now including **'Omar'**: **['Amr', 'Ali', 'Ezz', 'Omar']**.

In the fifth line, the code uses the **"extend"** method to add multiple elements to the list **"names"**. The specified elements **['Adel', 'Akl']** are appended to the end of the list. The **"extend"** method modifies the list in place. When you run this line, it will add **'Adel'** and **'Akl'** to the list.

In the sixth line, the code uses the **"print"** command to display the updated list **"names"**. When you run this line, it will print the modified list on the screen, now including **'Adel'** and **'Akl'**: **['Amr', 'Ali', 'Ezz', 'Omar', 'Adel', 'Akl']**.

In the seventh line, the code uses the **"remove"** method to remove an element from the list **"names"**. The method is given the index of the element to remove, in this case, **names[1]**, which refers to the element **'Ali'**. When you run this line, it will remove **'Ali'** from the list.

In the eighth line, the code uses the **"print"** command to display the updated list **"names"**. When you run this line, it will print the modified list on the screen, now without **'Ali'**: **['Amr', 'Ezz', 'Omar', 'Adel', 'Akl']**.

In the ninth line, the code uses the **"insert"** method to insert an element into the list **"names"**. The method is given the index where the element should be inserted, in this case, **1**, and the value **'Ali'**. When you run this line, it will insert **'Ali'** at **index 1**, shifting the other elements. The resulting list will be **['Amr', 'Ali', 'Ezz',**

```
'Omar', 'Adel', 'Akl'].
```

So, the output of this code will be:

```
['Amr', 'Ali', 'Ezz']  
['Amr', 'Ali', 'Ezz', 'Omar']  
['Amr', 'Ali', 'Ezz', 'Omar', 'Adel', 'Akl']  
['Amr', 'Ezz', 'Omar', 'Adel', 'Akl']
```

The code demonstrates different list operations: appending elements with "append", extending a list with "extend", removing elements with "remove", and inserting elements with "insert". These methods allow you to modify lists in various ways.

Q20. In the first line, the code uses the **"print"** command to display the value of the list **"names"**. When you run this line, it will print the list **"names"** on the screen: `['Amr', 'Ezz', 'Omar', 'Adel', 'Akl']`.

In the second line, the code creates a new list named **"names2"** by using the **"copy"** method on the existing list **"names"**. The **"copy"** method creates a **shallow copy** of the list, meaning it creates a new list object with the same elements as the original list. The new list **"names2"** will be an independent copy of **"names"** with the same elements.

When you run this code, it will create a new list **"names2"** that is a copy of **"names"**, preserving the elements and their order.

So, the output of this code will be:

```
['Amr', 'Ezz', 'Omar', 'Adel', 'Akl']
```

The code demonstrates how you can create a copy of an existing list using the **"copy"** method. The resulting copy will be a new list object that shares the same values as the original list.

Q21. In the first line, the code uses the **"clear"** method on the list **"names"** to remove all elements from the list. The **"clear"** method modifies the list in place by removing all its elements.

In the second line, the code uses the **"print"** command to display the value of the list **"names"**. When you run this line, it will print an **empty list**, as all elements have been removed: **[]**.

In the third line, the code uses the **"print"** command to display the value of the list **"names2"**. When you run this line, it will print the original list **"names2"**, which was created as a copy of **"names"** before it was cleared.

So, the output of this code will be:

```
[]  
['Amr', 'Ezz', 'Omar', 'Adel', 'Akl']
```

The first print statement shows an empty list after using the **"clear"** method on **"names"**. The second print statement shows the original list **"names2"**, which was created as a copy of **"names"** before it was cleared. Note that **"names2"** remains unchanged because **"clear"** only affects the list it is called on.

The code demonstrates how the **"clear"** method can be used to remove all elements from a list, leaving it empty.

Q22. In the first line, the code creates a tuple named **"family1"** that contains three elements: the strings **'Ahmed'**, **'Adel'**, and **'Amr'**.

In the second line, the code creates a tuple named **"family2"** that contains three elements: the strings **'Sarah'**, **'Hajer'**, and **'Rehab'**.

In the third line, the code creates a tuple named **"family3"** that contains three elements: the strings **'Tawfeek'**, **'Ezzat'**, and **'Foaad'**.

In the fourth line, the code creates a tuple named **"family4"** that contains four elements: the strings **'Hasan'**, **'Shokry'**, **'Ali'**, and **'Akl'**.

In the fifth line, the code creates a tuple named **"home1"** that contains two elements: the tuple **"family1"** and the tuple **"family2"**.

This creates a **nested tuple structure**.

In the sixth line, the code creates a tuple named **"home2"** that contains two elements: the tuple **"family3"** and the tuple **"family4"**. Again, this creates a **nested tuple structure**.

In the seventh line, the code uses indexing to access the second element (**'Adel'**) of the first element (**'family1'**) in the **"home1"** tuple. When you run this line, it will print **'Adel'** on the screen.

In the eighth line, the code uses indexing to access the first element (**'Tawfeek'**) of the second element (**'family3'**) in the **"home2"** tuple. When you run this line, it will print **'Tawfeek'** on the screen.

In the ninth line, the code uses indexing to access the first element (**'family1'**) in the **"home1"** tuple. When you run this line, it will print the tuple (**'Ahmed', 'Adel', 'Amr'**) on the screen.

In the tenth line, the code uses indexing to access the second element (**'family4'**) in the **"home2"** tuple. When you run this line, it will print the tuple (**'Hasan', 'Shokry', 'Ali', 'Ak1'**) on the screen.

In the eleventh line, the code uses the **"print"** command to display the entire **"home1"** tuple. When you run this line, it will print the nested tuple structure on the screen: (**'Ahmed', 'Adel', 'Amr'**), (**'Sarah', 'Hajer', 'Rehab'**)).

In the twelfth line, the code uses the **"print"** command to display the entire **"home2"** tuple. When you run this line, it will print the nested tuple structure on the screen: (**'Tawfeek', 'Ezzat', 'Foaad'**), (**'Hasan', 'Shokry', 'Ali', 'Ak1'**)).

So, the output of this code will be:

Adel

Tawfeek

(**'Ahmed', 'Adel', 'Amr'**)

(**'Hasan', 'Shokry', 'Ali', 'Ak1'**)

((**'Ahmed', 'Adel', 'Amr'**), (**'Sarah', 'Hajer', 'Rehab'**))

((**'Tawfeek', 'Ezzat', 'Foaad'**), (**'Hasan', 'Shokry', 'Ali', 'Ak1'**))

The code demonstrates how to create nested tuples, access elements within nested tuples using indexing, and print tuples and their nested structures. Tuples are immutable, meaning their elements cannot be changed once created.

Q23. The code defines six lists, **'family1'** to **'family6'**, each representing a different family. These lists contain the names of family members.

Then, the code creates two additional lists, **'home1'** and **'home2'**, which are **nested lists**. Each of these nested lists represents a collection of families.

In the next lines, the code uses indexing to access specific elements within the **nested lists**:

- **'home1[0][1]'** retrieves the element at **index 1** from the first family in **'home1'**, which is **'Adel'**.
- **'home1[1][0]'** retrieves the element at **index 0** from the second family in **'home1'**, which is **'Ehab'**.
- **'home2[0][2]'** retrieves the element at **index 2** from the first family in **'home2'**, which is **'Foaad'**.
- **'home2[2][3]'** retrieves the element at **index 3** from the third family in **'home2'**, which is **'Ak1'**.

The following lines print specific elements and entire lists:

- **'print(home1[0])'** prints the first family in **'home1'**: **['Ahmed', 'Adel', 'Amr']**.
- **'print(home2[1])'** prints the second family in **'home2'**: **['Abdelrahman', 'Abdelkareem']**.

After that, the code uses the **'del'** statement to remove elements from the lists:

- **'del home1[1]'** removes the second family from **'home1'**.
- **'del home2[1]'** removes the second family from **'home2'**.

Finally, the code prints the modified lists **'home1'** and **'home2'** without the removed families.

The output of the code will be:

Adel

Ehab

Foad

Akl

['Ahmed', 'Adel', 'Amr']

['Abdelrahman', 'Abdelkareem']

[['Ahmed', 'Adel', 'Amr'], ['Sarah', 'Hajer', 'Rehab']]

[['Tawfeek', 'Ezzat', 'Foad'], ['Hasan', 'Shokry', 'Ali', 'Akl']]

Q24. In the first line, the code creates a dictionary named **"person1"** with three key-value pairs: **'name'** with the value **'Amr'**, **'salary'** with the value **5000**, and **'active'** with the value **True**.

In the second line, the code creates a dictionary named **"person2"** with three key-value pairs: **'name'** with the value **'Ali'**, **'salary'** with the value **4000**, and **'active'** with the value **True**.

In the third line, the code creates a dictionary named **"person3"** with three key-value pairs: **'name'** with the value **'Ezz'**, **'salary'** with the value **3000**, and **'active'** with the value **True**.

In the fourth line, the code uses the **"print"** command to display the values of the key-value pairs in the **"person1"** dictionary. When you run this line, it will print: **'Amr 5000 True'**.

In the fifth line, the code uses the **"print"** command to display the values of the key-value pairs in the **"person2"** dictionary. When you run this line, it will print: **'Ali 4000 True'**.

In the sixth line, the code uses the **"print"** command to display the values of the key-value pairs in the **"person3"** dictionary. When you run this line, it will print: **'Ezz 3000 True'**.

So, the output of this code will be:

Amr 5000 True

Ali 4000 True

Ezz 3000 True

The code demonstrates how to create dictionaries and access values associated with specific keys. Dictionaries are data structures that store key-value pairs, allowing you to retrieve values based on their corresponding keys.

Q25. In the first line, the code creates a dictionary named **"person"** with three key-value pairs: **'name'** with the value **'Amr'**, **'salary'** with the value **5000**, and **'active'** with the value **True**.

In the second line, the code uses the **"keys"** method on the **"person"** dictionary to retrieve a view object containing all the keys in the dictionary. The **"keys"** method returns an iterable that represents the keys of the dictionary.

In the third line, the code uses the **"values"** method on the **"person"** dictionary to retrieve a view object containing all the values in the dictionary. The **"values"** method returns an iterable that represents the values of the dictionary.

In the fourth line, the code uses the **"print"** command to display the view object returned by the **"keys"** method. When you run this line, it will print the keys of the **"person"** dictionary.

In the fifth line, the code uses the **"print"** command to display the view object returned by the **"values"** method. When you run this line, it will print the values of the **"person"** dictionary.

So, the output of this code will be:

dict_keys(['name', 'salary', 'active'])

dict_values(['Amr', 5000, True])

The **"keys"** method returns a view object that represents the keys of the dictionary, and the **"values"** method returns a view object that

represents the values of the dictionary. These view objects can be iterated over or converted to other data structures if needed.

Q26. The `'list()'` function converts the view objects returned by `'person.keys()'` and `'person.values()'` into lists.

In the first line, the code converts the view object returned by `'person.keys()'` into a list using the `'list()'` function. This creates a list containing the keys of the dictionary.

In the second line, the code converts the view object returned by `'person.values()'` into a list using the `'list()'` function. This creates a list containing the values of the dictionary.

The `'list()'` function is used to convert the view objects into lists, which allows you to access the elements using indexing and perform other list operations.

So, the output of this code will be:

```
['name', 'salary', 'active']  
['Amr', 5000, True]
```

The `'list(person.keys())'` returns a list containing the keys of the dictionary, and `'list(person.values())'` returns a list containing the values of the dictionary.

Q27. The `'items()'` method on the `'person'` dictionary to retrieve a view object that contains tuples representing each key-value pair in the dictionary.

The `'items()'` method returns a view object that allows you to iterate over the key-value pairs of the dictionary. Each key-value pair is represented as a tuple, where the first element of the tuple is the key and the second element is the value.

In your code, you are using the `'print()'` function to display the view object returned by `'person.items()'`.

So, when you run this code, it will print the view object that represents the key-value pairs of the 'person' dictionary.

The output will look something like this:

```
dict_items([('name', 'Adel'), ('city', 'Giza'), ('salary', 3000)])
```

The '**dict_items**' view object is a specialised iterable that contains the key-value pairs of the dictionary. Each key-value pair is represented as a tuple, enclosed within the '**dict_items**' object.

If you want to convert the view object into a list or perform further operations on it, you can use the '**list()**' function to convert it into a list. For example:

```
person_items = list(person.items())  
print(person_items)
```

This will convert the view object into a list of tuples and print it:

```
[('name', 'Adel'), ('city', 'Giza'), ('salary', 3000)]
```

I hope this clarifies how the 'items()' method works and how to use it to retrieve the key-value pairs of a dictionary.

Q28. The '**items()**' method on the '**person**' dictionary to retrieve a view object that contains tuples representing each key-value pair in the dictionary. Then, you are using the '**list()**' function to convert the view object into a list.

When you run this code, it will print a list that contains tuples representing the key-value pairs of the '**person**' dictionary.

The output will look like this:

```
[('name', 'Adel'), ('city', 'Giza'), ('salary', 3000)]
```

In this case, the '**list()**' function converts the view object returned by '**person.items()**' into a list of tuples. Each tuple represents a key-value pair, where the first element is the key and the second element is the corresponding value.

By using '**list(person.items())**', you can retrieve a list of tuples containing the key-value pairs from the dictionary. This allows you to access and manipulate the key-value pairs as needed.

I hope this clarifies the code and how the '**list()**' function is used with the '**items()**' method to obtain the key-value pairs of a dictionary.

Q29. In the code you are creating an empty set called '**names**' using the curly braces '{}'.

Sets are unordered collections of unique elements. They can be created by enclosing elements within curly braces or by using the '**set()**' function.

In this case, you are using the '**set()**' function to create an empty set. The '**set()**' function returns a new empty set object.

When you run this code and print '**names**', it will display an empty set on the screen.

The output will look like this:

set()

The empty set is represented by '**set()**', indicating that there are no elements present in the set.

An empty set can be useful when you want to add elements to it later or when you want to perform set operations such as union, intersection, or difference with other sets.

I hope this clarifies the code and how an empty set is created and displayed using the '**set()**' function and '**print()**' command.

Q30. In the code you provided, you are creating a list called 'L1' with three elements: '**Ahmed**', '**Adel**', and '**Omar**'. Then, you are using the '**set()**' function to create a set called '**names**' from the elements

of the list.

Sets are unordered collections of unique elements, meaning they do not allow duplicate values. When you create a set using the `'set()'` function and pass a list as an argument, it will create a set containing only the unique elements from the list.

In this case, since the list `'L1'` contains three unique elements, the resulting set `'names'` will also contain those three elements: `'Ahmed'`, `'Adel'`, and `'Omar'`.

When you run this code and print `'names'`, it will display the set containing the unique elements from the list.

The output will look like this:

```
{'Ahmed', 'Adel', 'Omar'}
```

The elements of the set are enclosed within curly braces `'{}'` and separated by commas. Note that the order of elements in a set is not guaranteed because sets are unordered.

Using a set can be helpful when you want to work with unique elements and perform set operations such as union, intersection, or difference.

I hope this clarifies the code and how a set is created from a list using the `'set()'` function.

Q31. In the code you provided, you have two lists: `'L1'` and `'L2'`. You are creating a set called `'names'` from the elements of `'L1'` using the `'set()'` function.

Then, you are using the `'update()'` method to add the elements from `'L2'` to the `'names'` set. The `'update()'` method is used to update a set by adding elements from another iterable, such as a list, tuple, or another set.

In this case, the `'update()'` method adds the elements from `'L2'` to

the **'names'** set, resulting in a set that contains all the unique elements from both **'L1'** and **'L2'**.

When you run this code and print **'names'**, it will display the updated set that includes the unique elements from both lists.

The output will look like this:

```
{'Ahmed', 'Adel', 'Omar', 'Amr', 'Ali', 'Ezz'}
```

The **'names'** set now contains all the unique elements from both lists combined. Note that sets do not preserve the order of elements because they are unordered collections.

Using the **'update()'** method allows you to add elements to an existing set. It is a useful method when you want to combine multiple sets or add elements from another iterable to a set.

I hope this clarifies the code and how the **'update()'** method is used to add elements to a set from another iterable.

Q32. In the code you provided, you have a list **'L1'** containing two elements: **'Amr'** and **'Ali'**. You are creating a set called **'names'** from the elements of **'L1'** using the **'set()'** function.

Then, you are using the **'add()'** method to add the elements **'Ezz'** and **'Yaser'** to the **'names'** set. The **'add()'** method is used to add a single element to a set.

In this case, you are sequentially adding **'Ezz'** and **'Yaser'** to the **'names'** set using the **'add()'** method.

When you run this code and print **'names'**, it will display the updated set that includes the elements from **'L1'** as well as the elements you added using the **'add()'** method.

The output will look like this:

```
{'Amr', 'Ali', 'Ezz', 'Yaser'}
```

The **'names'** set now contains **'Amr'**, **'Ali'**, **'Ezz'**, and **'Yaser'**, as they have been added using the **'add()'** method. Note that sets are unordered collections, so the order of elements may vary in the output.

Using the **'add()'** method allows you to add individual elements to a set. It is a convenient method when you want to add elements one by one to an existing set.

I hope this clarifies the code and how the **'add()'** method is used to add elements to a set.

Q33. In the code you provided, you are creating an empty set called **'names'** using the **'set()'** function.

Then, you are using the **'add()'** method to add the element **'Ahmed'** to the **'names'** set three times.

However, since sets only store unique elements and do not allow duplicates, adding **'Ahmed'** multiple times will not result in duplicates in the set. The set will only contain a single occurrence of each unique element.

When you run this code and print **'names'**, it will display the set containing the unique element **'Ahmed'**.

The output will look like this:

```
{ 'Ahmed' }
```

The set **'names'** only contains the unique element **'Ahmed'** once, even though it was added multiple times. This demonstrates the behaviour of sets in eliminating duplicate elements.

Sets are useful when you want to store a collection of unique elements and perform set operations like union, intersection, and

difference.

I hope this clarifies the code and how sets handle duplicates by only storing unique elements.

Q34. In the code you provided, you have a list **'L1'** containing two elements: **'Amr'** and **'Ali'**. You are creating a set called **'names'** from the elements of **'L1'** using the **'set()'** function.

Then, you are using the **'add()'** method to add the elements **'Ezz'** and **'Yaser'** to the **'names'** set. The **'add()'** method is used to add a single element to a set.

Next, you are using the **'remove()'** method to remove the element **'Ali'** from the **'names'** set. The **'remove()'** method is used to remove a specific element from a set.

When you run this code and print **'names'**, it will display the updated set that includes the elements from **'L1'** and the elements you added using the **'add()'** method, but without the element **'Ali'** that was removed using the **'remove()'** method.

The output will look like this:

```
{'Amr', 'Ezz', 'Yaser'}
```

The **'names'** set now contains **'Amr'**, **'Ezz'**, and **'Yaser'**, as **'Ali'** was successfully removed using the **'remove()'** method. Note that sets are unordered collections, so the order of elements may vary in the output.

Using the **'remove()'** method allows you to remove a specific element from a set. It is useful when you want to selectively remove elements from a set based on certain criteria.

I hope this clarifies the code and how the **'remove()'** method is used to remove an element from a set.

Q35. In the code you provided, you have a list **'L1'** containing two

elements: **'Amr'** and **'Ali'**. You are creating a set called **'names'** from the elements of **'L1'** using the **'set()'** function.

Then, you are using the **'add()'** method to add the elements **'Ezz'** and **'Yaser'** to the **'names'** set. The **'add()'** method is used to add a single element to a set.

Next, you are using the **'discard()'** method to remove the element **'Ali'** from the **'names'** set. The **'discard()'** method removes a specific element from a set if it exists. If the element is not present, the method does nothing.

When you run this code and print **'names'**, it will display the updated set that includes the elements from **'L1'** and the elements you added using the **'add()'** method, but without the element **'Ali'** that was discarded using the **'discard()'** method.

The output will look like this:

```
{'Amr', 'Ezz', 'Yaser'}
```

The **'names'** set now contains **'Amr'**, **'Ezz'**, and **'Yaser'**, as **'Ali'** was successfully discarded using the **'discard()'** method. Note that sets are unordered collections, so the order of elements may vary in the output.

Using the **'discard()'** method allows you to remove a specific element from a set without raising an error if the element is not present. It is useful when you want to remove an element if it exists or perform the operation safely without interruption.

I hope this clarifies the code and how the **'discard()'** method is used to remove an element from a set.

Q36. In the given code:

1. **'L1 = ['Amr','Ali']'**: This line creates a list called **'L1'** containing two elements, **'Amr'** and **'Ali'**.

2. **'names = set(L1)'**: Here, we create a set called **'names'** by

converting the list `'L1'` into a set. A set is an unordered collection of unique elements, so any duplicates in the list will be automatically removed in the set. In this case, the set `'names'` will contain `'Amr'` and `'Ali'`.

3. `'names.add('Ezz')`: This line adds the string `'Ezz'` to the set `'names'`, so now `'names'` will contain `'Amr'`, `'Ali'`, and `'Ezz'`.

4. `'names.add('Yaser')`: This line adds the string `'Yaser'` to the set `'names'`, resulting in `'Amr'`, `'Ali'`, `'Ezz'`, and `'Yaser'`.

5. `'names.pop()'`: The `'pop()'` method is used to remove and return an arbitrary element from the set. Since sets are unordered, there is no guarantee about which element will be removed. In this case, one of the elements (`'Amr'`, `'Ali'`, `'Ezz'`, or `'Yaser'`) will be removed.

6. `'print(names)'`: Finally, this line prints the current content of the set `'names'` after the addition and removal operations.

Since sets are unordered, the output of `'print(names)'` could vary each time you run the code. It might look something like this: `'{'Ali', 'Ezz', 'Yaser'}'` or `'{'Amr', 'Ezz', 'Yaser'}'`. The exact order depends on which element was removed by the `'pop()'` method.

Q37. In this code, you are creating a set called `'names'`, adding some elements to it, and then using the `'clear()'` method to remove all items from the set. Let's break down the code step by step:

1. `'L1 = ['Amr','Ali']'`: This line creates a list called `'L1'` containing two elements, `'Amr'` and `'Ali'`.

2. `'names = set(L1)'`: Here, you create a set called `'names'` by converting the list `'L1'` into a set. A set is an unordered collection of unique elements, so any duplicates in the list will be automatically removed in the set. In this case, the set `'names'` will contain `'Amr'` and `'Ali'`.

3. `'names.add('Ezz')`: This line adds the string `'Ezz'` to the set

`'names'`, so now `'names'` will contain `'Amr'`, `'Ali'`, and `'Ezz'`.

4. `'names.add('Yaser')`: This line adds the string `'Yaser'` to the set `'names'`, resulting in `'Amr'`, `'Ali'`, `'Ezz'`, and `'Yaser'`.

5. `'names.clear()'`: The `'clear()'` method is used to remove all items from the set `'names'`, effectively emptying it.

6. `'print(names)'`: Finally, this line prints the current content of the set `'names'` after the addition and removal operations. Since you used `'names.clear()'` in the previous step, the output will be an empty set: `'set()'`.

Q38. In this code, you are creating two sets `'names1'` and `'names2'`, and then performing a union operation on them to combine their elements into a new set called `'all_names'`. Let's break down the code step by step:

1. `'names1 = set(['Adel','Omar','Atef'])'`: This line creates a set `'names1'` containing three elements: `'Adel'`, `'Omar'`, and `'Atef'`.

2. `'names2 = set(['Amr','Ali','Ezz'])'`: This line creates another set `'names2'` containing three elements: `'Amr'`, `'Ali'`, and `'Ezz'`.

3. `'all_names = names1.union(names2)'`: Here, you perform the union operation on sets `'names1'` and `'names2'`, using the `'union()'` method. The `'union()'` method combines the elements of both sets while removing any duplicates, resulting in a new set `'all_names'` that contains all the unique elements from both `'names1'` and `'names2'`.

4. `'print(all_names)'`: Finally, this line prints the content of the set `'all_names'`. The output will be the combined set of unique elements from `'names1'` and `'names2'`, which will be: `'{'Adel', 'Ali', 'Atef', 'Amr', 'Ezz', 'Omar'}'`.

Note that sets are unordered collections, so the order of elements in the output may vary from one execution to another. However, the elements themselves will remain the same.

Q39. In this code, you have two sets, `'names1'` and `'names2'`, and you are performing the union operation on them using the `'union()'` method. The union of two sets combines their elements, removing duplicates, and creates a new set called `'all_names'`. Let's break down the code:

1. `'names1 = set(['Adel','Omar','Atef','Amr'])'`: This line creates a set `'names1'` containing four elements: `'Adel'`, `'Omar'`, `'Atef'`, and `'Amr'`.

2. `'names2 = set(['Amr','Ali','Omar','Ezz','Adel'])'`: This line creates another set `'names2'` containing five elements: `'Amr'`, `'Ali'`, `'Omar'`, `'Ezz'`, and `'Adel'`.

3. `'all_names = names1.union(names2)'`: Here, you perform the union operation on sets `'names1'` and `'names2'` using the `'union()'` method. The `'union()'` method combines the elements of both sets while removing any duplicates. The resulting set `'all_names'` will contain all the unique elements from both `'names1'` and `'names2'`.

4. `'print(all_names)'`: Finally, this line prints the content of the set `'all_names'`. The output will be the combined set of unique elements from `'names1'` and `'names2'`, which will be: `'{'Adel', 'Ali', 'Atef', 'Amr', 'Ezz', 'Omar'}'`.

Note that sets are unordered collections, so the order of elements in the output may vary from one execution to another. However, the elements themselves will remain the same.

Q40. In this code, you have two sets, `'names1'` and `'names2'`, and you are performing the difference operation on them using the `'difference()'` method. The `'difference()'` method returns a new set containing the elements that are present in one set but not in the other. Let's break down the code:

1. `'names1 = set(['Adel','Omar','Atef','Amr'])'`: This line creates a set `'names1'` containing four elements: `'Adel'`, `'Omar'`, `'Atef'`, and

`'Amr'.`

2. `'names2 = set(['Amr','Ali','Omar','Ezz','Adel'])`: This line creates another set `'names2'` containing five elements: `'Amr'`, `'Ali'`, `'Omar'`, `'Ezz'`, and `'Adel'`.

3. `'all_names1 = names1.difference(names2)'`: Here, you perform the difference operation on sets `'names1'` and `'names2'` using the `'difference()'` method. The `'difference()'` method returns a new set that contains the elements present in `'names1'` but not in `'names2'`. The resulting set `'all_names1'` will be `'{Atef}'` since `'Atef'` is in `'names1'` but not in `'names2'`.

4. `'all_names2 = names2.difference(names1)'`: Similarly, this line performs the difference operation on sets `'names2'` and `'names1'`, creating a new set `'all_names2'` containing elements that are present in `'names2'` but not in `'names1'`. The resulting set `'all_names2'` will be `'{Ali, Ezz}'` since `'Ali'` and `'Ezz'` are in `'names2'` but not in `'names1'`.

5. `'print(all_names1)'`: This line prints the content of the set `'all_names1'`, which is `'{Atef}'`.

6. `'print(all_names2)'`: This line prints the content of the set `'all_names2'`, which is `'{Ali, Ezz}'`.

So, the final output of the code will be:

```
{'Atef'}  
{'Ali', 'Ezz'}
```

Q41. Let's understand the code:

1. The code defines a variable named `'name'` and assigns the string `'Ahmed Hassouna'` to it.

2. The code then uses the `'print()'` function to check if the character `'H'` is present in the `'name'` string.

3. The expression `'H' in name` is a membership test that checks if the character `'H'` is present in the string `name`. If the character `'H'` is present in the string, the expression will evaluate to `True`, otherwise, it will evaluate to `False`.

4. In this case, since the character `'H'` is present in the string `'Ahmed Hassouna'`, the expression will evaluate to `True`.

5. As a result, when you run the code, it will output:

`True`

This indicates that the character `'H'` is indeed present in the string `'Ahmed Hassouna'`.

Q42. In this code, you have a list called `names` containing three elements: `'Amr'`, `'Ali'`, and `'Ezz'`. The code then checks if the string `'Amr'` is present in the list using the `in` keyword. Let's break down the code:

1. `names = ['Amr','Ali','Ezz']`: This line creates a list called `names` containing three elements: `'Amr'`, `'Ali'`, and `'Ezz'`.

2. `print('Amr' in names)`: Here, you use the `in` keyword to check if the string `'Amr'` is present in the list `names`. The `in` operator returns a Boolean value (`True` or `False`) indicating whether the element is present in the list.

The output of the code will be `True`, as `'Amr'` is indeed present in the list `names`. If you were to check for an element that is not present in the list, the output would be `False`. For example, if you used `print('Yaser' in names)`, the output would be `False`, as `'Yaser'` is not in the list `names`.

Q43. In this code, you have a set called `names` containing three elements: `'Amr'`, `'Ali'`, and `'Ezz'`. The code then checks if the string `'Amr'` is present in the set using the `in` keyword. Let's understand the code:

1. `'names = {'Amr', 'Ali', 'Ezz}'`: This line creates a set called `'names'` containing three elements: `'Amr'`, `'Ali'`, and `'Ezz'`. Note that sets are defined using curly braces `'{'`.

2. `'print('Amr' in names)'`: Here, you use the `'in'` keyword to check if the string `'Amr'` is present in the set `'names'`. The `'in'` operator returns a Boolean value (`'True'` or `'False'`) indicating whether the element is present in the set.

The output of the code will be `'True'`, as `'Amr'` is indeed present in the set `'names'`. If you were to check for an element that is not present in the set, the output would be `'False'`. For example, if you used `'print('Yaser' in names)'`, the output would be `'False'`, as `'Yaser'` is not in the set `'names'`.

Q44. In this code, you have a tuple called `'names'` containing three elements: `'Amr'`, `'Ali'`, and `'Ezz'`. The code then checks if the string `'Amr'` is present in the tuple using the `'in'` keyword. Let's break down the code:

1. `'names = ('Amr', 'Ali', 'Ezz)'`: This line creates a tuple called `'names'` containing three elements: `'Amr'`, `'Ali'`, and `'Ezz'`. Tuples are defined using parentheses `'('`.

2. `'print('Amr' in names)'`: Here, you use the `'in'` keyword to check if the string `'Amr'` is present in the tuple `'names'`. The `'in'` operator returns a Boolean value (`'True'` or `'False'`) indicating whether the element is present in the tuple.

The output of the code will be `'True'`, as `'Amr'` is indeed present in the tuple `'names'`. If you were to check for an element that is not present in the tuple, the output would be `'False'`. For example, if you used `'print('Yaser' in names)'`, the output would be `'False'`, as `'Yaser'` is not in the tuple `'names'`.

Q45. In this code, you have a set called `'names'` containing three elements: `'Amr'`, `'Ali'`, and `'Ezz'`. The code then checks if the string

'Amr' is present in the set using the 'in' keyword. Let's break down the code:

1. `names = set(['Amr', 'Ali', 'Ezz'])`: This line creates a set called 'names' containing three elements: 'Amr', 'Ali', and 'Ezz'. The set is created by converting the list `['Amr', 'Ali', 'Ezz']` into a set using the `set()` function.

2. `print('Amr' in names)`: Here, you use the 'in' keyword to check if the string 'Amr' is present in the set 'names'. The 'in' operator returns a Boolean value ('True' or 'False') indicating whether the element is present in the set.

The output of the code will be 'True', as 'Amr' is indeed present in the set 'names'. If you were to check for an element that is not present in the set, the output would be 'False'. For example, if you used `print('Yaser' in names)`, the output would be 'False', as 'Yaser' is not in the set 'names'.

Q46. In this code, you have a variable 'my_dept' that contains the string 'Department 3', and you are checking if the character '3' is present in the string using the 'in' keyword. Let's break down the code:

1. `my_dept = 'Department 3'`: This line assigns the string 'Department 3' to the variable 'my_dept'.

2. `print('3' in my_dept)`: Here, you use the 'in' keyword to check if the character '3' is present in the string stored in the variable 'my_dept'. The 'in' operator returns a Boolean value ('True' or 'False') indicating whether the character or substring is present in the string.

The output of the code will be 'True', as the character '3' is indeed present in the string 'Department 3'. If you were to check for a different character or substring that is not present in the string, the output would be 'False'. For example, if you used `print('5' in my_dept)`, the output would be 'False', as '5' is not in the string

```
'Department 3'.
```

Q47. Let's go through the code step by step:

name = 'Amr': This line assigns the string **'Amr'** to the variable **'name'**.

print(name): The next line prints the value of the variable **'name'**, which is **'Amr'**.

del name: The **'del'** statement is used to delete the variable **'name'** from the current namespace. After this line executes, the variable **'name'** no longer exists.

So, when the code is executed, it will print:

Amr

After that, the variable **'name'** will be deleted from the namespace, and trying to access it again will result in an error.

Q48. In the provided code, a list named **'names'** is created, and then elements are added to it. The **'del'** statement is used to remove an element from the list. Let's go through the code step by step:

```
names = ['Amr', 'Ali', 'Ezz']  
print(names)
```

This part creates a list **'names'** containing three elements: **'Amr'**, **'Ali'**, and **'Ezz'**. The **'print'** statement is then used to display the entire list.

Output of the first print statement:

```
['Amr', 'Ali', 'Ezz']
```

```
del (names[2])  
print(names)
```

The **'del'** statement is used to delete the element at index 2 from the

list **'names'**, which is **'Ezz'**.

Output of the second print statement:

```
['Amr', 'Ali']
```

After executing the **'del'** statement, the element **'Ezz'** is removed from the list **'names'**, and the updated list is printed, which now contains only two elements: **'Amr'** and **'Ali'**.

Q49. Let's explain the code step by step:

1. Variables **'name1'**, **'name2'**, and **'name3'** are initialized with string values **'Ahmed'**, **'Adel'**, and **'Amr'**, respectively. These variables hold individual names.

2. **'names1'** is assigned a tuple **('Ahmed', 'Adel', 'Amr')**, which contains three names: **'Ahmed'**, **'Adel'**, and **'Amr'**.

3. **'names2'** is assigned a list **['Sarah', 'Hajer', 'Rehab', 'Heba']**, which contains four names: **'Sarah'**, **'Hajer'**, **'Rehab'**, and **'Heba'**.

4. The **'len()'** function is used to calculate the lengths of the strings in **'name1'**, **'name2'**, and **'name3'**, and these lengths are stored in the variables **'length_name1'**, **'length_name2'**, and **'length_name3'**, respectively.

5. Similarly, the **'len()'** function is used to calculate the number of elements (items) in **'names1'** and **'names2'**, and these lengths are stored in the variables **'length_names1'** and **'length_names2'**, respectively.

6. Finally, the **'print()'** statements display the results:

- **'print('length_name1:', length_name1, 'Characters')'** will output: **'length_name1: 5 Characters'**

- **'print('length_name2:', length_name2, 'Characters')'** will output: **'length_name2: 4 Characters'**

- **'print('length_name3:', length_name3, 'Characters')'** will


```
output: 'length_name3: 3 Characters'
```

```
- 'print('length_names1:', length_names1, 'Items')' will output:  
'length_names1: 3 Items'
```

```
- 'print('length_names1:', length_names2, 'Items')' will output:  
'length_names1: 4 Items'
```

These print statements display the lengths of the individual names ('name1', 'name2', and 'name3') in characters and the lengths of the lists 'names1' and 'names2' in terms of the number of items they contain.

Q50. In Python, **comments** are lines of code that are not executed and are only meant for developers to provide explanations or notes within the code. **They are ignored by the Python interpreter and serve as documentation or explanatory text for the code.**

In your code, you have used comments in various ways:

1. **Single-line comments:** These are comments that start with a hash symbol '#' and extend to the end of the line. They are used to add brief explanations or notes to specific lines of code.

Example:

```
name = 'Ahmed' # This is my name  
print(name) # Print name in here
```

In this example, the comments follow the '#' symbol and provide explanations for the variables and the print statement.

2. **Multi-line comments:** You have used both triple single quotes ('''') and triple double quotes ("""") to create multi-line comments. These are used when you need to add more extensive explanations that span multiple lines.

Example:

```
'''
```

```
And Can be multiline using triple single quote In First Paragraph  
And In Last Paragraph
```

```
'''
```

```
"""
```

**And Can be multiline using triple double quote In First Paragraph
And In the Last Paragraph**

```
"""
```

Although multi-line comments are not directly used in your code, they are a useful way to provide more detailed explanations for larger blocks of code.

Remember that the Python interpreter will not execute any code within comments, making them a great tool for documenting your code and making it more understandable to yourself and other developers.

Q51. Let's go through the code step by step, as it performs various mathematical operations on the variable `'num'`:

1. `'num = 5; print(num)'`: This line sets the value of the variable `'num'` to `'5'` and then prints the value of `'num'`. The output will be: `5`

2. `'num += 6; print(num)'`: This line increments the value of `'num'` by `'6'` (equivalent to `'num = num + 6'`) and then prints the updated value of `'num'`. The current value of `'num'` is `'5'`, so after this operation, `'num'` will be `'11'`. The output will be: `11`

3. `'num -= 4; print(num)'`: This line subtracts `'4'` from the current value of `'num'` and then prints the updated value. The current value of `'num'` is `'11'`, so after this operation, `'num'` will be `'7'`. The output will be: `7`

4. `'num *= 2; print(num)'`: This line multiplies the current value of `'num'` by `'2'` and then prints the updated value. The current value of `'num'` is `'7'`, so after this operation, `'num'` will be `'14'`. The output will be: `14`

5. `'num **= 2; print(num)'`: This line raises `'num'` to the power of `'2'` (i.e., `'num'` squared) and then prints the updated value. The

current value of `'num'` is `'14'`, so after this operation, `'num'` will be `'196'` (14^2). The output will be: **196**

6. `'num /= 3; print(num)'`: This line divides the current value of `'num'` by `'3'` and then prints the updated value. The current value of `'num'` is `'196'`, so after this operation, `'num'` will be approximately `'65.33333333'` (repeating decimal). The output will be:
65.33333333333333

7. `'num //= 3; print(num)'`: This line performs integer division on `'num'`, effectively removing the decimal part, and then prints the updated value. The current value of `'num'` is `'65.33333333'`, so after this operation, `'num'` will be `'65'` (the integer part of the result of the previous division). The output will be: **65**

8. `'num %= 12; print(num)'`: This line calculates the remainder of dividing `'num'` by `'12'` and then prints the updated value. The current value of `'num'` is `'65'`, so after this operation, `'num'` will be `'5'` (remainder of $65 / 12$). The output will be: **5**

That's the result of all the mathematical operations performed on the variable `'num'`.

Q52. Let's walk through the code step by step:

1. `'my_str = '''`: This line initializes an empty string and assigns it to the variable `'my_str'`.

2. `'my_str += "Hello"'`: This line appends the string `'"Hello"'` to the existing value of `'my_str'`. Since `'my_str'` was empty before this operation, it now becomes `'"Hello"'`.

3. `'my_str += " "'`: This line appends a space character to the existing value of `'my_str'`. Now, `'my_str'` becomes `'"Hello "'`. Note that we added a space between the words `"Hello"` and `"Ahmed"` to ensure there is a space between them when we concatenate the strings.

4. `'my_str += "Ahmed"'`: This line appends the string `'"Ahmed"'` to the existing value of `'my_str'`. Now, `'my_str'` becomes `'"Hello Ahmed"'`.

The space we added in the previous step ensures there is a space between **"Hello"** and **"Ahmed"** in the final string.

5. Finally, **'print(my_str)'**: This line prints the final value of **'my_str'**. The output will be: **Hello Ahmed**

This code demonstrates how you can build a string by concatenating multiple strings together using the **'+='** operator. It starts with an empty string and then adds **"Hello"**, a space, and **"Ahmed"** to it, resulting in the final string **"Hello Ahmed"**.

Q53. In this code, you have assigned the same value **'7'** to three different variables **'num1'**, **'num2'**, and **'num3'**. Let's go through it step by step:

1. **'num1 = num2 = num3 = 7'**: This line sets the value **'7'** to all three variables **'num1'**, **'num2'**, and **'num3'** simultaneously. This is achieved through what's known as **"chained assignment."** It means that the value **'7'** is assigned to **'num3'**, and then **'num2'** is assigned the same value as **'num3'**, and finally, **'num1'** is assigned the same value as **'num2'**. As a result, all three variables end up having the value **'7'**.

2. **'print(num1)'**: This line prints the value of **'num1'**, which is **'7'**. So the output will be: **7**

3. **'print(num2)'**: This line prints the value of **'num2'**, which is also **'7'**. Since **'num2'** was assigned the same value as **'num1'**, both variables have the value **'7'**. The output will be: **7**

4. **'print(num3)'**: This line prints the value of **'num3'**, which, once again, is **'7'**. As previously explained, **'num3'** was assigned the value **'7'** at the beginning, along with **'num1'** and **'num2'**. The output will be: **7**

So, the final output of the code will be:

7
7
7

All three variables, 'num1', 'num2', and 'num3', hold the value '7', as they were all assigned to it in the same statement.

Q54. Let's understand the code step by step:

1. **Comments:** Comments are non-executable lines in Python that provide explanations or annotations. They start with a hash symbol (#) and are ignored by the Python interpreter.

2. **Single-Line and Multi-Line Comments:**

- **Single-line** comments begin with a hash symbol and extend only to the end of the line.

- **Multi-line** comments can be achieved using triple single quotes ("""") or triple double quotes (""") at the beginning and end.

3. Variable and Printing:

- **'name = 'Ahmed':** Assigns the string 'Ahmed' to the variable 'name'.

- **'print(name)'**: Prints the value of the variable 'name', which is 'Ahmed'.

- **'num = 5; print(num)'**: Initializes 'num' to 5 and then prints its value.

- **'num += 6; print(num)'**: Increments 'num' by 6 and prints its new value.

- **'num -= 4; print(num)'**: Decrements 'num' by 4 and prints its new value.

- **'num *= 2; print(num)'**: Multiplies 'num' by 2 and prints its new value.

- **'num **= 2; print(num)'**: Raises 'num' to the power of 2 and prints its new value.

- **'num /= 3; print(num)'**: Divides 'num' by 3 and prints its new value.

- **'num //= 3; print(num)'**: Performs floor division on 'num' by 3 and prints its new value (**rounds down to the nearest whole number**).

- **'num %= 12; print(num)'**: Calculates the remainder of 'num' divided by 12 and prints its new value.

4. String Concatenation:

- `'my_str = ""`: Initialises an empty string named `'my_str'`.
- `'my_str += "Hello"`: Appends the string `"Hello"` to `'my_str'`.
- `'my_str += " "`: Appends a space to `'my_str'`.
- `'my_str += "Ahmed"`: Appends the string `"Ahmed"` to `'my_str'`.
- `'print(my_str)'`: Prints the final value of `'my_str'`, which will be `"Hello Ahmed"`.

5. Variable Assignment:

- `'num1 = num2 = num3 = 7'`: Assigns the value `7` to all three variables `'num1'`, `'num2'`, and `'num3'`.

6. Concatenating Strings and Numbers:

- `'num = 99'`: Assigns the value `99` to the variable `'num'`.
- `'print('My number is: ' + str(num))'`: Concatenates the string `'My number is: '` with the string representation of `'num'` (converted using `'str()'`), and then prints the result.

Note: The provided code will not result in any errors and should work as expected when executed.

Q55. Let's break down the code step by step:

```
num_1 = '77'
num_2 = '77.33'
```

In this part of the code, two variables are defined: `'num_1'` and `'num_2'`. They both store string values. `'num_1'` is assigned the string `'77'`, and `'num_2'` is assigned the string `'77.33'`.

```
num1 = int(num_1)
num2 = float(num_2)
```

Here, the `'int()'` and `'float()'` functions are used to convert the strings into their respective numeric data types. Let's see the conversion process for each:

- `'int(num_1)'`: The `'int()'` function is applied to `'num_1'`, which contains the string `'77'`. The `'int()'` function converts the string

to an integer data type. The resulting value, '77', is stored in the variable 'num1'.

- `'float(num_2)'`: The `'float()'` function is applied to `'num_2'`, which contains the string `'77.33'`. The `'float()'` function converts the string to a floating-point number data type. The resulting value, `'77.33'`, is stored in the variable `'num2'`.

```
print(num1, type(num1))
print(num2, type(num2))
```

Finally, the `'print()'` function is used to display the values of `'num1'` and `'num2'`, along with their corresponding data types.

- `'print(num1, type(num1))'`: This line prints the value of `'num1'`, which is `'77'`, along with its data type. The output will be `'77 <class 'int'>'`.

- `'print(num2, type(num2))'`: This line prints the value of `'num2'`, which is `'77.33'`, along with its data type. The output will be `'77.33 <class 'float'>'`.

By using the `'int()'` and `'float()'` functions, the code successfully converts the strings `'77'` and `'77.33'` into numeric data types, specifically an **integer** and a **floating-point** number, respectively. The `'print()'` statements display the converted values along with their corresponding data types.

Q56. Let's go through the code step by step:

```
num1 = 1
num2 = 0
```

In this part of the code, two variables are defined: `'num1'` and `'num2'`. They are assigned the integer values `'1'` and `'0'`, respectively.

```
bool1 = bool(num1)
```

```
bool2 = bool(num2)
```

Here, the `'bool()'` function is used to convert the integers `'num1'` and `'num2'` into their corresponding boolean values. The `'bool()'` function returns `'True'` if the argument is nonzero (**not equal to 0**) and `'False'` if the argument is zero.

- `'bool1 = bool(num1)'`: The `'bool()'` function is applied to `'num1'`, which is `'1'`. Since `'1'` is nonzero, the boolean value `'True'` is stored in the variable `'bool1'`.

- `'bool2 = bool(num2)'`: The `'bool()'` function is applied to `'num2'`, which is `'0'`. Since `'0'` is zero, the boolean value `'False'` is stored in the variable `'bool2'`.

```
print(bool1, type(bool1))
print(bool2, type(bool2))
```

Finally, the `'print()'` function is used to display the boolean values of `'bool1'` and `'bool2'`, along with their corresponding data types.

- `'print(bool1, type(bool1))'`: This line prints the value of `'bool1'`, which is `'True'`, along with its data type. The output will be `'True <class 'bool'>'`.

- `'print(bool2, type(bool2))'`: This line prints the value of `'bool2'`, which is `'False'`, along with its data type. The output will be `'False <class 'bool'>'`.

By using the `'bool()'` function, the code successfully converts the integers `'1'` and `'0'` into their respective boolean values `'True'` and `'False'`. The `'print()'` statements display the converted values along with their corresponding data types, which are of type `'bool'`.

Q57. In this code, the variable `'c'` is assigned the character `'A'`. The `'ord()'` function is then used to obtain the Unicode code point of the character `'A'`, and the result is printed.

Let's break down the code step by step:

```
c = 'A'
```

In this line, the variable '**c**' is assigned the character '**A**'.

```
i = ord(c)
```

Here, the '**ord()**' function is applied to the variable '**c**', which contains the character '**A**'. The '**ord()**' function returns the Unicode code point of the given character. For the character '**A**', the Unicode code point is **65 (in decimal)**.

```
print(i)
```

Finally, the '**print()**' function is used to display the value of the variable '**i**', which holds the Unicode code point of the character '**A**'.

Output: **65**

The code successfully prints the Unicode code point of the character '**A**', which is '**65**'. Unicode is a standard that assigns a unique numeric value to every character in most of the world's writing systems, facilitating the representation of various languages and symbols in computers. In this case, the character '**A**' corresponds to the Unicode code point **65**.

Q58. In this code, the variable '**i**' is assigned the integer value '**65**'. The '**chr()**' function is then used to obtain the character representation of the Unicode code point '**65**', and the result is printed.

Let's break down the code step by step:

```
i = 65
```

In this line, the variable '**i**' is assigned the integer value '**65**'.

```
c = chr(i)
```

Here, the '**chr()**' function is applied to the variable '**i**', which contains the integer '**65**'. The '**chr()**' function takes an integer (Unicode code point) as an argument and returns the corresponding

character. For the Unicode code point '65', the character representation is 'A'.

```
print(c)
```

Finally, the 'print()' function is used to display the value of the variable 'c', which holds the character representation of the Unicode code point '65'.

Output: A

The code successfully prints the character 'A', which corresponds to the Unicode code point '65'. The 'chr()' function allows us to convert Unicode code points into their corresponding characters, making it easier to work with character-based data.

Q59. In this code, a 'range' object is created using the 'range()' function with an argument of '5'. The 'range()' function generates a sequence of numbers starting from 0 (by default) up to, but not including, the specified number (5 in this case). Then, the code prints the individual elements of the 'range' object by accessing them using indexing.

Let's break down the code step by step:

```
r = range(5)
```

In this line, the 'range()' function is used to create a 'range' object 'r' with the argument '5'. The 'range()' function generates a sequence of numbers '[0, 1, 2, 3, 4]'. Note that the end value (5) is not included in the sequence.

```
print(r[0], r[1], r[2], r[3], r[4])
```

The 'print()' function is used to display the individual elements of the 'range' object 'r' by accessing them using indexing. In Python, indexing starts from 0, so 'r[0]' corresponds to the first element in the 'range', which is '0'. 'r[1]' corresponds to the second element ('1'), 'r[2]' corresponds to the third element ('2'), and so on.

Output: 0 1 2 3 4

The code successfully prints the individual elements of the **'range'** object **'r'**, which are **'0', '1', '2', '3', and '4'**. This demonstrates how to create and access elements in a **'range'** object using indexing.

Q60. In this code, a **'range'** object is created using the **'range()'** function with an argument of **'7'**. The **'range()'** function generates a sequence of numbers starting from 0 (by default) up to, but not including, the specified number (**7 in this case**). Then, the code prints the individual elements of the **'range'** object by accessing them using indexing.

Let's break down the code step by step:

```
r = range(7)
```

In this line, the **'range()'** function is used to create a **'range'** object **'r'** with the argument **'7'**. The **'range()'** function generates a sequence of numbers **'[0, 1, 2, 3, 4, 5, 6]'**. Note that the end value (**7**) is not included in the sequence.

```
print(r[0], r[1], r[2], r[3], r[4], r[5], r[6])
```

The **'print()'** function is used to display the individual elements of the **'range'** object **'r'** by accessing them using indexing. In Python, indexing starts from 0, so **'r[0]'** corresponds to the first element in the **'range'**, which is **'0'**. **'r[1]'** corresponds to the second element (**'1'**), **'r[2]'** corresponds to the third element (**'2'**), and so on until **'r[6]'** corresponds to the seventh element (**'6'**).

Output: **0 1 2 3 4 5 6**

The code successfully prints the individual elements of the **'range'** object **'r'**, which are **'0', '1', '2', '3', '4', '5', and '6'**. This demonstrates how to create and access elements in a **'range'** object using indexing.

Q61. In this code, a **'range'** object is created using the **'range()'** function with the arguments **'1'** and **'8'**. The **'range()'** function generates a sequence of numbers starting from the first argument (inclusive) up to, but not including, the second argument (exclusive). Then, the code prints the individual elements of the

'range' object by accessing them using indexing.

Let's break down the code step by step:

```
r = range(1, 8)
```

In this line, the 'range()' function is used to create a 'range' object 'r' with the arguments '1' and '8'. The 'range()' function generates a sequence of numbers '[1, 2, 3, 4, 5, 6, 7]'. Note that the end value ('8') is not included in the sequence.

```
print(r[0], r[1], r[2], r[3], r[4], r[5], r[6])
```

The 'print()' function is used to display the individual elements of the 'range' object 'r' by accessing them using indexing. In Python, indexing starts from 0, so 'r[0]' corresponds to the first element in the 'range', which is '1'. 'r[1]' corresponds to the second element ('2'), 'r[2]' corresponds to the third element ('3'), and so on until 'r[6]' corresponds to the seventh element ('7').

Output: 1 2 3 4 5 6 7

The code successfully prints the individual elements of the 'range' object 'r', which are '1', '2', '3', '4', '5', '6', and '7'. This demonstrates how to create and access elements in a 'range' object using indexing.

Q62. In this code, a 'range' object is created using the 'range()' function with the arguments '1', '11', and '2'. The 'range()' function generates a sequence of numbers starting from the first argument (inclusive), up to the second argument (exclusive), with a step size specified by the third argument (2 in this case). Then, the code prints the individual elements of the 'range' object by accessing them using indexing.

Let's break down the code step by step:

```
r = range(1, 11, 2)
```

In this line, the 'range()' function is used to create a 'range' object 'r' with the arguments '1', '11', and '2'. The 'range()' function generates a sequence of numbers '[1, 3, 5, 7, 9]'. Note that

the end value ('11') is not included in the sequence. The step size of '2' means that the sequence increments by '2' at each step.

```
print(r[0], r[1], r[2], r[3], r[4])
```

The '**print()**' function is used to display the individual elements of the '**range**' object '**r**' by accessing them using indexing. In Python, indexing starts from 0, so '**r[0]**' corresponds to the first element in the '**range**', which is '1'. '**r[1]**' corresponds to the second element ('3'), '**r[2]**' corresponds to the third element ('5'), and so on until '**r[4]**' corresponds to the fifth element ('9').

Output: 1 3 5 7 9

The code successfully prints the individual elements of the '**range**' object '**r**', which are '1', '3', '5', '7', and '9'. This demonstrates how to create and access elements in a '**range**' object using indexing with a specified step size.

Q63. In this code, a '**range**' object is created using the '**range()**' function with the arguments '2', '21', and '3'. The '**range()**' function generates a sequence of numbers starting from the first argument (inclusive), up to the second argument (exclusive), with a step size specified by the third argument (3 in this case). Then, the code prints the individual elements of the '**range**' object by accessing them using indexing.

Let's break down the code step by step:

```
r = range(2, 21, 3)
```

In this line, the '**range()**' function is used to create a '**range**' object '**r**' with the arguments '2', '21', and '3'. The '**range()**' function generates a sequence of numbers '[2, 5, 8, 11, 14, 17, 20]'. Note that the end value ('21') is not included in the sequence. The step size of '3' means that the sequence increments by '3' at each step.

```
print(r[0], r[1], r[2], r[3], r[4], r[5], r[6])
```

The '**print()**' function is used to display the individual elements of

the **'range'** object **'r'** by accessing them using indexing. In Python, indexing starts from 0, so **'r[0]'** corresponds to the first element in the **'range'**, which is **'2'**. **'r[1]'** corresponds to the second element (**'5'**), **'r[2]'** corresponds to the third element (**'8'**), and so on until **'r[6]'** corresponds to the seventh element (**'20'**).

Output: **2 5 8 11 14 17 20**

The code successfully prints the individual elements of the **'range'** object **'r'**, which are **'2'**, **'5'**, **'8'**, **'11'**, **'14'**, **'17'**, and **'20'**. This demonstrates how to create and access elements in a **'range'** object using indexing with a specified step size.

Q64

In this code, a **'range'** object is created using the **'range()'** function with the arguments **'ord('A')'** and **'ord('Z')+1'**. The **'range()'** function generates a sequence of numbers starting from the Unicode code point of the character **'A'** (inclusive) up to the Unicode code point of the character **'Z'** (inclusive). Then, the code prints the individual characters corresponding to the Unicode code points obtained from the **'range'** object by accessing them using indexing.

Let's break down the code step by step:

```
r = range(ord('A'), ord('Z')+1)
```

In this line, the **'range()'** function is used to create a **'range'** object **'r'** with the arguments **'ord('A')'** and **'ord('Z')+1'**. The **'ord()'** function is used to get the Unicode code points of the characters **'A'** and **'Z'**. So, the **'range'** object **'r'** will contain a sequence of integers representing the Unicode code points from the code point of **'A'** to the code point of **'Z'**.

For example, the Unicode code point of **'A'** is **65**, and the Unicode code point of **'Z'** is **90**. The **'range'** object will contain integers from **65** to **90** (both inclusive), representing the Unicode code points of characters from **'A'** to **'Z'**.

```
print(chr(r[0]),chr(r[1]),chr(r[2]), chr(r[3]), chr(r[4]), chr(r[5]))
```

The '**print()**' function is used to display the individual characters corresponding to the Unicode code points obtained from the '**range**' object '**r**' by accessing them using indexing. In Python, indexing starts from 0, so '**r[0]**' corresponds to the first element in the '**range**', which is the code point of '**A**'. '**chr(r[0])**' converts the code point back to the character representation, which is '**A**'.

Similarly, '**chr(r[1])**' converts the code point of the next element in the '**range**' to its corresponding character representation ('**B**'). This process continues for the subsequent elements until '**chr(r[5])**', which corresponds to the character representation of the code point of '**F**'.

Output: **A B C D E F**

The code successfully prints the characters '**A**', '**B**', '**C**', '**D**', '**E**', and '**F**', which are obtained from the '**range**' object '**r**' representing the sequence of Unicode code points from the code point of '**A**' to the code point of '**Z**'. This demonstrates how to use the '**ord()**' function to get Unicode code points and the '**chr()**' function to convert code points back to characters.

Q65. In this code, a '**range**' object is created using the '**range()**' function with the arguments '**6**', '**1**', and '**-1**'. The '**range()**' function generates a sequence of numbers starting from the first argument (inclusive) and going towards the second argument (exclusive) in reverse order, with a step size specified by the third argument ('**-1**' in this case). Then, the code prints the individual elements of the '**range**' object by accessing them using indexing.

Let's break down the code step by step:

```
python
```

```
r = range(6, 1, -1)
```

In this line, the '**range()**' function is used to create a '**range**'

object 'r' with the arguments '6', '1', and '-1'. The 'range()' function generates a sequence of numbers '[6, 5, 4, 3, 2]'. Note that the start value ('6') is included in the sequence, but the end value ('1') is not included. The step size of '-1' means that the sequence decrements by '1' at each step.

```
python
print(r[0], r[1], r[2], r[3], r[4])
```

The 'print()' function is used to display the individual elements of the 'range' object 'r' by accessing them using indexing. In Python, indexing starts from 0, so 'r[0]' corresponds to the first element in the 'range', which is '6'. 'r[1]' corresponds to the second element ('5'), 'r[2]' corresponds to the third element ('4'), and so on until 'r[4]' corresponds to the fifth element ('2').

Output:

```
6 5 4 3 2
```

The code successfully prints the individual elements of the 'range' object 'r', which are '6', '5', '4', '3', and '2'. This demonstrates how to create and access elements in a 'range' object using indexing with a specified step size in reverse order.

Q66. In this code, a 'range' object is created using the 'range()' function with the arguments '5', '0', and '-1'. The 'range()' function generates a sequence of numbers starting from the first argument (inclusive) and going towards the second argument (exclusive) in reverse order, with a step size specified by the third argument ('-1' in this case). Then, the code prints the individual elements of the 'range' object by accessing them using indexing.

Let's break down the code step by step:

```
python
```



```
r = range(5, 0, -1)
```

In this line, the `'range()'` function is used to create a `'range'` object `'r'` with the arguments `'5'`, `'0'`, and `'-1'`. The `'range()'` function generates a sequence of numbers `'[5, 4, 3, 2, 1]'`. Note that the start value (`'5'`) is included in the sequence, but the end value (`'0'`) is not included. The step size of `'-1'` means that the sequence decrements by `'1'` at each step.

```
python
print(r[0], r[1], r[2], r[3], r[4])
```

The `'print()'` function is used to display the individual elements of the `'range'` object `'r'` by accessing them using indexing. In Python, indexing starts from 0, so `'r[0]'` corresponds to the first element in the `'range'`, which is `'5'`. `'r[1]'` corresponds to the second element (`'4'`), `'r[2]'` corresponds to the third element (`'3'`), and so on until `'r[4]'` corresponds to the fifth element (`'1'`).

Output:

```
5 4 3 2 1
```

The code successfully prints the individual elements of the `'range'` object `'r'`, which are `'5'`, `'4'`, `'3'`, `'2'`, and `'1'`. This demonstrates how to create and access elements in a `'range'` object using indexing with a specified step size in reverse order. The sequence starts from `'5'` and decrements by `'1'` at each step until it reaches `'1'`, but `'0'` is not included in the sequence.

Q67. In this code, a `'range'` object is created using the `'range()'` function with the arguments `'10'`, `'0'`, and `'-2'`. The `'range()'` function generates a sequence of numbers starting from the first argument (inclusive) and going towards the second argument (exclusive) in reverse order, with a step size specified by the third

argument ('-2' in this case). Then, the code prints the individual elements of the 'range' object by accessing them using indexing.

Let's break down the code step by step:

```
python
r = range(10, 0, -2)
```

In this line, the 'range()' function is used to create a 'range' object 'r' with the arguments '10', '0', and '-2'. The 'range()' function generates a sequence of numbers '[10, 8, 6, 4, 2]'. Note that the start value ('10') is included in the sequence, but the end value ('0') is not included. The step size of '-2' means that the sequence decrements by '2' at each step.

```
python
print(r[0], r[1], r[2], r[3], r[4])
```

The 'print()' function is used to display the individual elements of the 'range' object 'r' by accessing them using indexing. In Python, indexing starts from 0, so 'r[0]' corresponds to the first element in the 'range', which is '10'. 'r[1]' corresponds to the second element ('8'), 'r[2]' corresponds to the third element ('6'), and so on until 'r[4]' corresponds to the fifth element ('2').

Output:

```
10 8 6 4 2
```

The code successfully prints the individual elements of the 'range' object 'r', which are '10', '8', '6', '4', and '2'. This demonstrates how to create and access elements in a 'range' object using indexing with a specified step size in reverse order. The sequence starts from '10' and decrements by '2' at each step until it reaches '2', but '0' is not included in the sequence.

Q68. The provided code is almost correct, but there is an issue with the variable declaration. The variable "name" should have a value assigned before it can be used. Let's fix the code and explain how it works:

```
python
name = input("Enter your name:")
print('Hello ' + name)
```

Explanation:

1. 'name = input("Enter your name:")': This line uses the 'input()' function to prompt the user to enter their name. The 'input()' function reads the user's input from the console, and the entered value is stored in the variable 'name'.

2. 'print('Hello ' + name)': This line uses the 'print()' function to display the message "Hello " followed by the value stored in the 'name' variable. The '+' operator is used to concatenate the strings "Hello " and the user's name, resulting in a greeting that says "Hello" followed by the name the user entered.

When you run the code, it will ask you to enter your name in the console. After entering your name and pressing the Enter key, it will print the greeting message "Hello" followed by your name. For example, if you enter "John", it will print "Hello John".

Q69. In this code, the 'input()' function is used to prompt the user to enter any name. The entered value is then stored in the variable 'name'. After that, the 'type()' function is used to determine the data type of the variable 'name'. Let's break down the code:

```
python
name = input('Enter any name:')
```

In this line, the 'input()' function is called with the prompt

message 'Enter any name:'. The program will wait for the user to enter a name, and the entered value will be stored in the variable 'name'.

```
python  
print(type(name))
```

Here, the 'type()' function is applied to the variable 'name'. The 'type()' function returns the data type of the given variable. In this case, it will print the data type of the variable 'name'.

When you run the code and enter a name, let's say "John", it will display the data type of the variable 'name' on the console:

```
<class 'str'>
```

In this example, the entered name "John" is a string, so the output shows that the data type of the variable 'name' is '<class 'str'>', which means it's a string type. The 'input()' function always reads user input as a string, so the 'name' variable will be of type 'str' regardless of the data entered by the user.

Q70. In this code, the user is prompted to enter two numbers. The entered numbers are converted to integers using the 'int()' function, and then the sum of the two numbers is calculated and printed. Let's go through the code step by step:

```
python  
num1 = int(input('Enter number 1:'))
```

In this line, the 'input()' function is used to prompt the user to enter a number. The entered value will be read as a string. Then, the 'int()' function is applied to convert the entered string into an integer, and the result is stored in the variable 'num1'.

```
python
num2 = int(input('Enter number 2:'))
```

Similarly, in this line, the user is prompted to enter another number. The entered value is read as a string, and the 'int()' function converts it into an integer, which is then stored in the variable 'num2'.

```
python
result = num1 + num2
```

In this line, the sum of 'num1' and 'num2' is calculated using the '+' operator, and the result is stored in the variable 'result'.

```
python
print(result)
```

Finally, the 'print()' function is used to display the value of the variable 'result', which contains the sum of the two numbers entered by the user.

When you run the code, it will prompt you to enter two numbers. For example, if you enter '5' and '7', it will calculate the sum ('5 + 7 = 12') and print the result:

```
Enter number 1: 5
Enter number 2: 7
12
```

The code successfully takes two numbers as input, converts them to integers, calculates their sum, and then prints the result.

Q71. In this code, the 'random' module is imported, and the 'randint()' function from the 'random' module is used to generate random integers between 1 and 10 (inclusive). The 'print()' function is then used to display each randomly generated number. Let's go through the code step by step:

```
python
import random
```

In this line, the 'random' module is imported, allowing us to access various functions related to random number generation.

```
python
num = random.randint(1, 10)
print(num)
```

In these lines, the 'randint()' function is used to generate a random integer between 1 and 10 (both inclusive), and the result is stored in the variable 'num'. The 'print()' function is then used to display the value of 'num'.

The above two lines of code are repeated multiple times to generate and print five random integers between 1 and 10:

Output (example):

```
8
4
10
3
9
```

Each time the code is executed, it will generate five different random integers within the specified range (1 to 10) and display them using the 'print()' function.

The `'random.randint(a, b)'` function generates a random integer between 'a' and 'b' (inclusive). In this case, 'a' is 1, and 'b' is 10, so the function will generate random integers from 1 to 10.

Q72. In this code, the `'randint()'` function from the `'random'` module is imported, and it is used to generate random integers within specific ranges. The `'print()'` function is then used to display each randomly generated number. Let's go through the code step by step:

```
python
from random import randint
```

In this line, the `'randint()'` function is imported directly from the `'random'` module. This allows us to use `'randint()'` without prefixing it with `'random.'`.

```
python
num = randint(10, 20)
print(num)
```

In these lines, the `'randint()'` function is used to generate a random integer between 10 and 20 (both inclusive), and the result is stored in the variable `'num'`. The `'print()'` function is then used to display the value of `'num'`.

The above two lines of code are repeated multiple times to generate and print six random integers within the specified ranges:

Output (example):

```
17
27
37
47
52
64
```

Each time the code is executed, it will generate different random integers within the specified ranges (10 to 20, 20 to 30, 30 to 40, and so on) and display them using the 'print()' function.

The 'randint(a, b)' function generates a random integer between 'a' and 'b' (inclusive). In this case, different ranges are used for each call to 'randint()', resulting in random integers within those specific ranges being printed.

Q73. In this code, logical AND operations are performed on boolean values, and the results are printed. The logical AND operator ('and') returns 'True' only if both operands are 'True'. Otherwise, it returns 'False'. Let's go through the code step by step:

```
python
bool1 = True and True
print('true AND true =', bool1)
```

In these lines, the logical AND operation is performed between 'True' and 'True'. Since both operands are 'True', the result of the logical AND is 'True', which is stored in the variable 'bool1'. The 'print()' function is then used to display the result with a corresponding message.

```
python
bool2 = False and True
print('false AND true =', bool2)
```

Here, the logical AND operation is performed between 'False' and 'True'. Since one of the operands ('False') is 'False', the result of the logical AND is 'False', which is stored in the variable 'bool2'. The 'print()' function is then used to display the result with a corresponding message.


```
python
bool3 = True and False
print('true AND false =', bool3)
```

In these lines, the logical AND operation is performed between 'True' and 'False'. Again, since one of the operands ('False') is 'False', the result of the logical AND is 'False', which is stored in the variable 'bool3'. The 'print()' function is then used to display the result with a corresponding message.

```
python
bool4 = False and False
print('false AND false =', bool4)
```

In this case, the logical AND operation is performed between 'False' and 'False'. Since both operands are 'False', the result of the logical AND is 'False', which is stored in the variable 'bool4'. The 'print()' function is then used to display the result with a corresponding message.

Output:

```
true AND true = True
false AND true = False
true AND false = False
false AND false = False
```

The code successfully performs logical AND operations on boolean values and prints the results accordingly.

Q74. In this code, various logical operations are performed on boolean values, and the results are printed. The logical operators used are 'and', 'or', and 'not'. Let's go through the code step by step:

```
python
bool5 = True and True and False and True
print('T&T&F&T =', bool5)
```

In these lines, the logical AND operation is performed between four boolean values: 'True', 'True', 'False', and 'True'. Since one of the operands ('False') is 'False', the result of the logical AND is 'False', which is stored in the variable 'bool5'. The 'print()' function is then used to display the result with a corresponding message.

```
python
bool6 = True and True and True and True
print('T&T&T&T =', bool6)
```

In this case, the logical AND operation is performed between four boolean values, all of which are 'True'. Since all operands are 'True', the result of the logical AND is 'True', which is stored in the variable 'bool6'. The 'print()' function is then used to display the result with a corresponding message.

```
python
bool7 = True or True
print('true OR true =', bool7)
```

Here, the logical OR operation is performed between two boolean values, both of which are 'True'. Since at least one of the operands ('True') is 'True', the result of the logical OR is 'True', which is stored in the variable 'bool7'. The 'print()' function is then used to display the result with a corresponding message.

Similarly, logical OR operations are performed between various combinations of boolean values in the subsequent lines.

```
python
bool14 = not True
```

```
print('NOT true =', bool14)
```

In this line, the logical NOT operation is performed on the boolean value 'True'. The NOT operator negates the value, so 'not True' results in 'False', which is stored in the variable 'bool14'. The 'print()' function is then used to display the result with a corresponding message.

Output:

```
T&T&F&T = False
T&T&T&T = True
true OR true = True
false OR true = True
true OR false = True
false OR false = False
F|F|F|F = False
F|F|T|F = True
T|T|T|T = True
NOT true = False
```

The code successfully performs various logical operations on boolean values and prints the results accordingly.

Q75. In this code, comparison operations are performed between the variables 'x' and 'y', which have been assigned the values '7' and '9', respectively. The results of these comparison operations are then printed. Let's go through the code step by step:

```
python
x, y = 7, 9
```

In this line, two variables 'x' and 'y' are initialized with the values '7' and '9', respectively, using tuple unpacking.

```
python
b1 = x > y
print(b1)
```

In these lines, the comparison operation 'x > y' is performed. The result of this comparison is 'False' because '7' is not greater than '9'. The result ('False') is stored in the variable 'b1', and then it is printed.

```
python
b2 = x < y
print(b2)
```

Here, the comparison operation 'x < y' is performed. The result of this comparison is 'True' because '7' is less than '9'. The result ('True') is stored in the variable 'b2', and then it is printed.

```
python
b3 = x >= y
print(b3)
```

In these lines, the comparison operation 'x >= y' is performed. The result of this comparison is 'False' because '7' is not greater than or equal to '9'. The result ('False') is stored in the variable 'b3', and then it is printed.

```
python
b4 = x <= y
print(b4)
```

In these lines, the comparison operation 'x <= y' is performed. The result of this comparison is 'True' because '7' is less than or equal to '9'. The result ('True') is stored in the variable 'b4', and then it is printed.

```
python
b5 = x == y
print(b5)
```

Here, the comparison operation 'x == y' is performed. The result of this comparison is 'False' because '7' is not equal to '9'. The result ('False') is stored in the variable 'b5', and then it is printed.

```
python
b6 = x != y
print(b6)
```

In these lines, the comparison operation 'x != y' is performed. The result of this comparison is 'True' because '7' is not equal to '9'. The result ('True') is stored in the variable 'b6', and then it is printed.

Output:

```
False
True
False
True
False
True
```

The code successfully performs various comparison operations between 'x' and 'y' and prints the results accordingly.

Q76. In this code, different number representations are used and converted between various numeric bases, such as decimal, hexadecimal, octal, and binary. The built-in Python functions 'hex()', 'oct()', and 'bin()' are used to convert decimal numbers to their respective representations. The 'int()' function is used to

convert numbers from other bases back to decimal. Let's go through the code step by step:

```
python
num_d = 255
```

In this line, a decimal number '255' is assigned to the variable 'num_d'.

```
python
num_h = 0xff
```

Here, a hexadecimal number '0xff' is assigned to the variable 'num_h'. In Python, hexadecimal numbers are represented with a '0x' prefix.

```
python
num_o = 0o377
```

In these lines, an octal number '0o377' is assigned to the variable 'num_o'. In Python, octal numbers are represented with a '0o' prefix.

```
python
num_b = 0b11111111
```

In this line, a binary number '0b11111111' is assigned to the variable 'num_b'. In Python, binary numbers are represented with a '0b' prefix.

```
python
print('Hexadecimal:', hex(num_d))
print('Octal:', oct(num_d))
print('Binary:', bin(num_d))
```

These lines use the 'hex()', 'oct()', and 'bin()' functions to convert the decimal number 'num_d' (which is '255') to its respective hexadecimal, octal, and binary representations. The results are then printed.

```
python
print('Decimal from h:', int(num_h))
print('Decimal from o:', int(num_o))
print('Decimal from b:', int(num_b))
```

These lines use the 'int()' function to convert the numbers 'num_h', 'num_o', and 'num_b' from their respective bases (hexadecimal, octal, and binary) back to decimal. The results are then printed.

Output:

```
Hexadecimal: 0xff
Octal: 0o377
Binary: 0b1111111
Decimal from h: 255
Decimal from o: 255
Decimal from b: 255
```

The code successfully demonstrates conversions between different number bases and displays the results accordingly.

Q77. In this code, the 'format()' function is used to convert numbers between different numeric bases, such as decimal, hexadecimal, octal, and binary. The 'format()' function allows us to specify the desired format for the output. Let's go through the code step by step:

```
python
num_d = 255
num_h = 0xff
num_o = 0o377
num_b = 0b1111111
```

In these lines, four variables are assigned with values representing the number '255' in different bases: 'num_d' for decimal, 'num_h' for hexadecimal, 'num_o' for octal, and 'num_b' for binary.

```
python
print('Hexadecimal:', format(num_d, 'x'))
print('Octal:', format(num_d, 'o'))
print('Binary:', format(num_d, 'b'))
```

These lines use the 'format()' function to convert the decimal number 'num_d' (which is '255') to its respective hexadecimal, octal, and binary representations. The 'format()' function is called with two arguments: the number to be formatted ('num_d') and the format specifier ('x', 'o', and 'b' for hexadecimal, octal, and binary, respectively). The results are then printed.

```
python
print('Decimal from h:', format(num_h, 'd'))
print('Decimal from o:', format(num_o, 'd'))
print('Decimal from b:', format(num_b, 'd'))
```

These lines use the 'format()' function to convert the numbers 'num_h', 'num_o', and 'num_b' from their respective bases (hexadecimal, octal, and binary) back to decimal. The 'format()' function is called with the format specifier 'd', which indicates decimal format. The results are then printed.

Output:

```
Hexadecimal: ff
Octal: 377
Binary: 11111111
Decimal from h: 255
Decimal from o: 255
Decimal from b: 255
```


The code successfully demonstrates conversions between different number bases using the 'format()' function and displays the results accordingly.

Q78. In this code, the 'int()' function is used to convert numbers represented as strings in different bases (binary, octal, decimal, and hexadecimal) back to decimal integers. Let's go through the code step by step:

```
python
print(int('11111111', 2))
```

In this line, the string '11111111' represents a binary number. The 'int()' function is used with the second argument '2', which specifies that the given string is in base 2 (binary). The 'int()' function then converts the binary string '11111111' to its equivalent decimal integer, which is '255', and prints it.

```
python
print(int('377', 8))
```

Here, the string '377' represents an octal number. The 'int()' function is used with the second argument '8', indicating that the given string is in base 8 (octal). The 'int()' function converts the octal string '377' to its equivalent decimal integer, which is also '255', and prints it.

```
python
print(int('255', 10))
```

In these lines, the string '255' represents a decimal number. The 'int()' function is used with the second argument '10', indicating that the given string is already in base 10 (decimal). Since the base

is the same as the target base, the 'int()' function does not perform any conversion and simply returns the integer value '255', which is then printed.

```
python
print(int('ff', 16))
```

In this line, the string 'ff' represents a hexadecimal number. The 'int()' function is used with the second argument '16', which indicates that the given string is in base 16 (hexadecimal). The 'int()' function then converts the hexadecimal string 'ff' to its equivalent decimal integer, which is '255', and prints it.

Output:

```
255
255
255
255
```

The code successfully converts the numbers represented as strings in different bases back to decimal integers using the 'int()' function and displays the results accordingly.

Q79. In this code, various slicing operations are performed on the string 'my', and different substrings are printed. The slicing syntax 'my[start:stop]' is used to extract a portion of the string. The 'start' index is inclusive, and the 'stop' index is exclusive. If the 'start' index is not specified, it defaults to 0, and if the 'stop' index is not specified, it defaults to the end of the string. Let's go through the code step by step:

```
python
my = 'Welcome to Hassouna Academy'
```

In this line, the string `'Welcome to Hassouna Academy'` is assigned to the variable `'my'`.

```
python
print(my[0], my[1], my[2], my[3], my[4], my[5], my[6])
```

Here, individual characters of the string `'my'` are accessed using indexing. The index numbers `'0'` to `'6'` correspond to the first seven characters of the string. The characters are printed one by one with spaces between them.

Output:

W e l c o m e

```
python
print(my[:7])
```

In this line, slicing is used to extract the substring from index `'0'` to index `'6'`. The `'start'` index is not specified, so it defaults to `'0'`, and the `'stop'` index is `'7'`. The substring `'Welcome'` is printed.

```
python
print(my[0:7])
```

This line is equivalent to the previous one. It also slices the substring from index `'0'` to index `'6'` (exclusive), resulting in the substring `'Welcome'`.

```
python
print(my[11:])
```

In this line, slicing is used to extract the substring starting from

index '11' until the end of the string. The substring 'Hassouna Academy' is printed.

```
python
print(my[:-len(my) + 7])
```

Here, slicing is used to extract the substring from the beginning of the string until the '7th' character from the end. The 'len(my)' gives the length of the string (which is '28' characters), and 'len(my) - 7' gives the index of the '7th' character from the end. By slicing from the beginning of the string until that index (exclusive), the substring 'Welcome to Hassouna' is printed.

```
python
print(my[11:len(my)])
```

In this line, slicing is used to extract the substring starting from index '11' until the end of the string. The 'len(my)' gives the length of the string (which is '28' characters), so this slice is equivalent to 'print(my[11:])'. The substring 'Hassouna Academy' is printed.

Output:

```
Welcome
Welcome
Hassouna Academy
Welcome to Hassouna
Hassouna Academy
```

The code successfully demonstrates various slicing operations on the string 'my' and prints different substrings as specified.

Q80. In this code, a string 'str_names' containing names separated by semicolons (';') is split into a list of names using the 'split()'

method. The 'split()' method is called on the string with the separator as the argument, and it returns a list of substrings. Let's go through the code step by step:

```
python
str_names = 'Ahmed;Adel;Amr;Ali;Omar;Haitham'
```

In this line, a string 'Ahmed;Adel;Amr;Ali;Omar;Haitham' is assigned to the variable 'str_names'.

```
python
list_names = str_names.split(';')
```

In this line, the 'split()' method is used on the string 'str_names' with the separator ';'. The 'split()' method separates the string into substrings whenever it encounters the semicolon ';' character. The resulting substrings are then stored as elements in a list, which is assigned to the variable 'list_names'.

```
python
print(str_names)
```

This line prints the original string 'str_names', which contains names separated by semicolons.

Output:

```
Ahmed;Adel;Amr;Ali;Omar;Haitham
```

```
python
print(list_names)
```

This line prints the list 'list_names', which contains individual names as separate elements in the list.

Output:

```
['Ahmed', 'Adel', 'Amr', 'Ali', 'Omar', 'Haitham']
```

The code successfully splits the string 'str_names' into individual names using semicolons as separators and stores them as separate elements in the list 'list_names'. The original string and the resulting list are printed to demonstrate the split operation.

Q81. In this code, the 'join()' method is used to concatenate the characters of the string 'str1' with a hyphen ('-') as a separator. The 'join()' method takes an iterable (e.g., a string, list, tuple) as an argument and returns a new string where the elements of the iterable are joined together using the specified separator. Let's go through the code step by step:

```
python
str1 = 'Hello'
```

In this line, the string 'Hello' is assigned to the variable 'str1'.

```
python
str2 = '-'.join(str1)
```

In this line, the 'join()' method is used on the string 'str1' with the separator '-'. Since 'str1' is a string, the 'join()' method treats it as an iterable of characters. The method joins each character of 'str1' with a hyphen ('-') in between and creates a new string, which is assigned to the variable 'str2'.

```
python
print(str1)
```

This line prints the original string 'str1', which is 'Hello'.

Output:

Hello

```
python
print(str2)
```

This line prints the new string 'str2', which is the result of joining the characters of 'str1' with hyphens in between.

Output:

H-e-l-l-o

The 'join()' method separates each character of the original string 'str1' with a hyphen, effectively creating a new string with hyphens between each letter. This is why the output of 'str2' shows the characters of 'str1' separated by hyphens. The original string 'str1' remains unchanged.

Q82. In this code, a list of names 'list_names' is joined into a single string using the 'join()' method. The 'join()' method takes an iterable (e.g., a list, tuple, or string) as an argument and returns a new string where the elements of the iterable are joined together using the specified separator. Let's go through the code step by step:

```
python
list_names = ['Amr', 'Ali', 'Ezz']
```

In this line, a list '["Amr", "Ali", "Ezz"]' is assigned to the

```
variable 'list_names'.
```

```
python  
str_names = ';'.join(list_names)
```

In this line, the 'join()' method is used on the list 'list_names' with the separator ';'. The method joins each element of the list into a single string, using semicolons (;) as separators. The resulting string is assigned to the variable 'str_names'.

```
python  
print(list_names)
```

This line prints the original list 'list_names', which contains three names.

Output:

```
['Amr', 'Ali', 'Ezz']
```

```
python  
print(str_names)
```

This line prints the new string 'str_names', which is the result of joining the elements of 'list_names' with semicolons in between.

Output:

```
Amr;Ali;Ezz
```

The 'join()' method concatenates the elements of the list 'list_names' into a single string, separated by semicolons (;). The resulting string 'str_names' shows the names from the list separated by semicolons. The original list 'list_names' remains unchanged.

Q83. In this code, a list of names 'list_names' is joined into a single string using the 'join()' method, with a newline character ('\n') as the separator. The 'join()' method takes an iterable (e.g., a list, tuple, or string) as an argument and returns a new string where the elements of the iterable are joined together using the specified separator. The newline character '\n' is used to insert line breaks between the names when they are joined into the new string. Let's go through the code step by step:

```
python
list_names = ['Amr', 'Ali', 'Ezz', 'Ehab']
```

In this line, a list ['Amr', 'Ali', 'Ezz', 'Ehab'] is assigned to the variable 'list_names'.

```
python
str_names = '\n'.join(list_names)
```

In this line, the 'join()' method is used on the list 'list_names' with the separator '\n', which represents the newline character. The method joins each element of the list into a single string, inserting a newline character ('\n') between each name. The resulting string is assigned to the variable 'str_names'.

```
python
print(list_names)
```

This line prints the original list 'list_names', which contains four names.

Output:

```
['Amr', 'Ali', 'Ezz', 'Ehab']
```

```
python
print(str_names)
```

This line prints the new string 'str_names', which is the result of joining the elements of 'list_names' with newline characters ('\n') in between.

Output:

```
Amr
Ali
Ezz
Ehab
```

The 'join()' method concatenates the elements of the list 'list_names' into a single string, inserting newline characters ('\n') between the names. As a result, the names are displayed on separate lines in the output. The original list 'list_names' remains unchanged.

Q84. In this code, the string formatting operator '%' is used to create a new string 'my' by combining the greeting message 'Hello ' with the name 'Amr'. The '%s' placeholder in the format string 'Hello %s' is replaced with the value of the variable 'name', resulting in the final greeting message. Let's go through the code step by step:

```
python
name = 'Amr'
```

In this line, the string 'Amr' is assigned to the variable 'name'.

```
python
my = 'Hello %s' % name
```

In this line, the string `'Hello %s'` is used as a format string. The `'%s'` placeholder acts as a placeholder for a string value. The string formatting operator `'%'` is used to replace the `'%s'` placeholder with the value of the variable `'name'` (`'Amr'`). The resulting string is assigned to the variable `'my'`.

```
python
print(my)
```

This line prints the string `'my'`, which contains the greeting message `'Hello '` followed by the name `'Amr'`.

Output:

```
Hello Amr
```

The code successfully combines the greeting message `'Hello '` with the name `'Amr'` using string formatting and prints the final greeting message.

Q85. In this code, the string formatting operator `'%'` is used to create a new string `'my'` that displays the addition of two numbers (`'num1'` and `'num2'`) along with their respective values. The `'%d'` placeholders in the format string are replaced with the integer values of `'num1'`, `'num2'`, and the result of their addition. Let's go through the code step by step:

```
python
num1 = 7
num2 = 9
```

In these lines, two integer variables `'num1'` and `'num2'` are assigned the values `'7'` and `'9'`, respectively.

```
python
```

```
my = '%d + %d = %d' % (num1, num2, num1 + num2)
```

In this line, the string `'7 + 9 = %d'` is used as a format string. The `'%d'` placeholders act as placeholders for integer values. The string formatting operator `'%'` is used to replace the `'%d'` placeholders with the values of `'num1'`, `'num2'`, and the result of their addition (`'num1 + num2'`, which is `'16'`). The resulting string is assigned to the variable `'my'`.

```
python
```

```
print(my)
```

This line prints the string `'my'`, which contains the equation `'7 + 9 = 16'`.

Output:

```
7 + 9 = 16
```

The code successfully creates a string displaying the addition of `'num1'` and `'num2'` along with their respective values using string formatting and prints the resulting equation.

Q86. In this code, you are using the `'%'` operator to format a string, specifically, to insert the ASCII character corresponding to the decimal value `'65'` into another string. Let's break it down step by step:

1. `'my = '65 is ASCII for %c' % 'A'`: In this line, you have a string `'65 is ASCII for %c'`, where `'%c'` is a placeholder for a character. The `'%'` operator is then used to format this string. The value `'A'` is specified after the `'%'` operator, which represents the character you want to insert into the string.

2. `'%c'` in the string `'65 is ASCII for %c'` will be replaced by the character specified after the `'%'` operator, which is `'A'`.

3. In ASCII, the decimal value `'65'` represents the capital letter `'A'`.

4. When you run the code, the `'print()'` function will output the following:

```
65 is ASCII for A
```

The `'%c'` placeholder was replaced with the character `'A'`, and the final output indicates that the ASCII value `'65'` corresponds to the letter `'A'`.

Keep in mind that the `'%'` operator is an older style of string formatting in Python. The newer and recommended way to format strings is by using f-strings (available since Python 3.6) or the `'str.format()'` method. If you are using Python 3.6 or later, consider using f-strings for string formatting, as they offer a more concise and readable syntax.

Q87. It looks like you are trying to format a string using various format specifiers. However, there's a slight mistake in the code. Let's correct it and go through it step by step:

```
python
my = 'Character %c'
my += '\nString %s'
my += '\nDecimal %d'
my += '\nInteger %i'
my += '\nexponent %e'
my += '\nExponent %E'
my += '\nFloat %f'
my += '\nFloat %0.2f'
```

```
my += '\nNumber %g,%g'
my += '\nHexadecimal %x' % 65
print(my)
```

1. In the first part of the code, you are building a string named 'my' and appending various format specifiers to it using the '+' operator.

2. '%c' is a format specifier used to represent a single character. You don't provide any values after it, so it will not be replaced with any character yet.

3. '%s' is a format specifier used to represent a string. Similarly, you don't provide any value after it.

4. '%d' and '%i' are format specifiers used to represent decimal integers. Again, you haven't provided any values after them.

5. '%e' and '%E' are format specifiers used to represent numbers in exponential notation. No values are given after them.

6. '%f' is a format specifier used to represent floating-point numbers. '%0.2f' is used to represent a floating-point number with two decimal places.

7. '%g' is a format specifier used to represent numbers in either standard or exponential notation, depending on the value. You've provided two of them with a comma ',' between them, but no values yet.

8. '%x' is a format specifier used to represent an integer in hexadecimal notation. You provided the value '65' after it, so it will be replaced by the hexadecimal representation of '65', which is '41'.

Now, there seems to be a mistake in the code where you have '% 'A'', '% 'Hi'', etc. These are not used correctly. To format strings using the format specifiers, you should use the '%(value)' syntax. Let's

correct it:

```
python
my %= ('A', 'Hi', 55.99, 77, 33, 33, 99.77)
print(my)
```

The corrected code uses the '%' operator along with parentheses '()' to format the string 'my'. The values 'A', 'Hi', '55.99', '77', '33', '33', and '99.77' are passed in the parentheses in order, and they will replace the respective format specifiers in 'my'.

Now, let's explain what the output will be:

```
Character A
String Hi
Decimal 55
Integer 77
exponent 3.399000e+01
Exponent 3.399000E+01
Float %f
Float 99.77
Number 99.77,99.77
Hexadecimal 41
```

Note: The '%f' specifier was not provided with a value, so it is not replaced in the output. Also, '%g' is used in the format string but not provided with a value, so it remains as is in the output.

As you can see, the format specifiers are replaced with the respective values you provided in parentheses. The '%e' and '%E' specifiers represent numbers in exponential notation, and the '%f' specifier represents floating-point numbers. The '%x' specifier was provided with the value '65', so it was replaced with the hexadecimal representation of '65', which is '41'.

I hope this clarifies the usage of format specifiers and the

corrected code. If you have any further questions or need more clarification, feel free to ask!

Q88. In the provided code snippet, there are two variables defined:

1. 'name': This variable is assigned the string value 'Ahmed'. It stores the name "Ahmed" as a string.

2. 'say_hello': This variable is assigned the string value 'Hello {my}'. The string contains a placeholder '{my}', which is intended to be replaced with another value later.

To summarize:

- 'name' contains the string 'Ahmed'.
- 'say_hello' contains the string 'Hello {my}'.

It seems like there might be a missing part in the code where '{my}' is intended to be replaced with the value stored in the 'name' variable. If you want to print a greeting with the name variable, you can use string formatting or concatenation. For example:

Using string formatting:

```
python
name = 'Ahmed'
say_hello = 'Hello {my}'.format(my=name)
print(say_hello)
```

Using f-string (Python 3.6+):

```
python
name = 'Ahmed'
say_hello = f'Hello {name}'
print(say_hello)
```

Using concatenation:

```
python
```



```
name = 'Ahmed'
say_hello = 'Hello ' + name
print(say_hello)
```

All three methods will print: 'Hello Ahmed'.

Q89. In the updated code snippet, you are using the '.format()' method on the 'say_hello' string to replace the '{my}' placeholder with the value stored in the 'name' variable, and then printing the result.

Here's how it works:

1. 'say_hello': 'Hello {my}'
2. 'name': 'Ahmed'
3. 'my_format': The result of calling '.format()' on 'say_hello' with the value of 'name' replacing '{my}'.

Let's see the output:

```
python
name = 'Ahmed'
say_hello = 'Hello {my}'
my_format = say_hello.format(my=name)
print(my_format)
```

The output will be:

Hello Ahmed

In this code, the '{my}' placeholder in the 'say_hello' string is replaced with the value of 'name', which is 'Ahmed'. The result is stored in the 'my_format' variable, and when you print it, you see the greeting message, "Hello Ahmed."

Q90. In this code snippet, you are using the `'.format()'` method to format a string containing placeholders for three variables: `'n1'`, `'n2'`, and `'r'`. The `'.format()'` method replaces these placeholders with the values specified in the `'.format()'` function call, and then the formatted string is stored in the `'my_format'` variable and printed.

Let's break it down step by step:

1. `'n1 = 7'`: This assigns the value `'7'` to the variable `'n1'`.
2. `'n2 = 3'`: This assigns the value `'3'` to the variable `'n2'`.
3. `'r = 7 + 3'`: This calculates the sum of `'7'` and `'3'`, which is `'10'`, and assigns the result to the variable `'r'`.

Now, the `'.format()'` method is applied to the string `'{n1} + {n2} = {r}'`:

- `'{n1}'` is replaced with the value of `'n1'`, which is `'7'`.
- `'{n2}'` is replaced with the value of `'n2'`, which is `'3'`.
- `'{r}'` is replaced with the value of `'r'`, which is `'10'` (since `'7 + 3 = 10'`).

The formatted string is then stored in the `'my_format'` variable.

Finally, you print the value of `'my_format'`, which will display:

```
7 + 3 = 10
```

So, the output of the code will be `'7 + 3 = 10'`, which represents the result of the arithmetic operation using the values specified in the `'.format()'` function call.

Q91. In this code snippet, you are working with two strings, `'str1'` and `'str2'`, and applying the `'.lower()'` and `'.upper()'` methods to them. These methods are used to change the case of the characters in

a string. Let's see the code step by step:

```
python
str1 = 'HELLO'
str2 = 'welcome'
```

1. 'str1' is assigned the value 'HELLO'.
2. 'str2' is assigned the value 'welcome'.

Next, you use the '.lower()' and '.upper()' methods:

```
python
print(str1.lower())
print(str2.upper())
```

1. 'str1.lower()': This method converts all characters in the string 'str1' to lowercase. The output of this line will be:

```
hello
```

2. 'str2.upper()': This method converts all characters in the string 'str2' to uppercase. The output of this line will be:

```
WELCOME
```

So, the final output of the entire code will be:

```
hello
WELCOME
```

As you can see, the '.lower()' method changes all uppercase

characters in 'str1' to lowercase, and the '.upper()' method changes all lowercase characters in 'str2' to uppercase.

Q92. The code snippet contains several examples of string methods in Python. Let's go through each line and explain what each method does:

```
python
print('HELLO'.isupper())
```

Output: 'True'

Explanation: The 'isupper()' method returns 'True' if all characters in the string are uppercase, and in this case, the entire string 'HELLO' is in uppercase.

```
python
print('hello'.islower())
```

Output: 'True'

Explanation: The 'islower()' method returns 'True' if all characters in the string are lowercase, and in this case, the entire string 'hello' is in lowercase.

```
python
print('HEllo'.isalpha())
```

Output: 'True'

Explanation: The 'isalpha()' method returns 'True' if all characters in the string are alphabetic (letters), and there are no non-alphabetic characters in the string 'HEllo'.

```
python
print('ABC45'.isalnum())
```

Output: 'True'

Explanation: The 'isalnum()' method returns 'True' if all characters in the string are alphanumeric (letters or digits), and there are no non-alphanumeric characters in the string 'ABC45'.

```
python
print('12345'.isdigit())
```

Output: 'True'

Explanation: The 'isdigit()' method returns 'True' if all characters in the string are digits (numeric characters), and there are no non-digit characters in the string '12345'.

```
python
print(' '.isspace())
```

Output: 'True'

Explanation: The 'isspace()' method returns 'True' if all characters in the string are whitespace characters (spaces, tabs, newlines, etc.). In this case, the string contains only a space character.

```
python
print('1 AB@'.isprintable())
```

Output: 'False'

Explanation: The 'isprintable()' method returns 'True' if all characters in the string are printable (can be displayed on the screen), and there are no control characters. In this case, the string contains a non-printable character, '@'.

Please note that these methods return boolean values ('True' or 'False') based on the conditions they check in the strings.

Q93. Let's go through each line of the code snippet and see the output for each 'isupper()', 'islower()', 'isalpha()', 'isalnum()', 'isdigit()', 'isspace()', and 'isprintable()' methods applied to the respective strings:

```
python
print('HeLLo'.isupper())
```

Output: 'False'

Explanation: The 'isupper()' method returns 'True' if all characters

in the string are uppercase. In this case, the string `'HeLLO'` contains both uppercase and lowercase characters, so the result is `'False'`.

```
python
print('hEllo'.islower())
```

Output: `'False'`

Explanation: The `'islower()'` method returns `'True'` if all characters in the string are lowercase. In this case, the string `'hEllo'` contains both uppercase and lowercase characters, so the result is `'False'`.

```
python
print('HE7lo'.isalpha())
```

Output: `'False'`

Explanation: The `'isalpha()'` method returns `'True'` if all characters in the string are alphabetic (letters), and there are no non-alphabetic characters. In this case, the string `'HE7lo'` contains the digit `'7'`, so the result is `'False'`.

```
python
print('AB@45'.isalnum())
```

Output: `'False'`

Explanation: The `'isalnum()'` method returns `'True'` if all characters in the string are alphanumeric (letters or digits), and there are no non-alphanumeric characters. In this case, the string `'AB@45'` contains the special character `'@'`, so the result is `'False'`.

```
python
print('12A45'.isdigit())
```

Output: `'False'`

Explanation: The `'isdigit()'` method returns `'True'` if all characters in the string are digits (numeric characters), and there are no non-digit characters. In this case, the string `'12A45'` contains the letter `'A'`, so the result is `'False'`.

```
python
print(' . '.isspace())
```

Output: 'False'

Explanation: The 'isspace()' method returns 'True' if all characters in the string are whitespace characters (spaces, tabs, newlines, etc.). In this case, the string ' . ' contains non-whitespace characters, so the result is 'False'.

```
python
print('\n'.isprintable())
```

Output: 'False'

Explanation: The 'isprintable()' method returns 'True' if all characters in the string are printable (can be displayed on the screen), and there are no control characters. In this case, the string '\n' contains the newline character, which is a control character, so the result is 'False'.

In summary, these methods return boolean values based on the conditions they check in the respective strings.

Q94. In this code snippet, you have a string 'my' and you are using the '.find()' method to find the index of a specific substring within the 'my' string. The '.find()' method returns the index of the first occurrence of the substring in the original string. If the substring is not found, it returns -1. Let's go through the code step by step:

```
python
my = 'Hello Amr and Welcome Back Amr'
indexFind1 = my.find('amr')
indexFind2 = my.find('Amr')
print(indexFind1)
print(indexFind2)
```

1. 'my = 'Hello Amr and Welcome Back Amr''': This assigns the string

`'Hello Amr and Welcome Back Amr'` to the variable `'my'`.

2. `indexFind1 = my.find('amr')`: This line uses the `'.find()'` method to search for the lowercase substring `'amr'` in the `'my'` string. The method is case-sensitive, so it won't find the uppercase `'Amr'`. Since the lowercase `'amr'` occurs in the string at index 6 (starting from 0-based indexing), the value of `indexFind1` will be `'6'`.

3. `indexFind2 = my.find('Amr')`: This line uses the `'.find()'` method to search for the uppercase substring `'Amr'` in the `'my'` string. It finds the first occurrence of `'Amr'` at index 12, and therefore the value of `indexFind2` will be `'12'`.

4. `print(indexFind1)`: This prints the value of `indexFind1`, which is `'6'`.

5. `print(indexFind2)`: This prints the value of `indexFind2`, which is `'12'`.

So, the output of the code will be:

```
6
12
```

As mentioned earlier, the `'.find()'` method returns the index of the first occurrence of the substring, and it's case-sensitive. Therefore, `'amr'` and `'Amr'` are treated as different substrings in this context.

Q95. In this code snippet, you have a string `'my'`, and you are using the `'.find()'` method to find the index of the substring `'Welcome'` within the `'my'` string. Once you find the index, you use slicing to extract the portion of the string starting from that index until the end of the string. Let's go through the code step by step:

python


```
my = 'Hello Amr and Welcome Back Amr'
i = my.find('Welcome')
print(my[i:])
```

1. 'my = 'Hello Amr and Welcome Back Amr'': This assigns the string 'Hello Amr and Welcome Back Amr' to the variable 'my'.

2. 'i = my.find('Welcome')': This line uses the '.find()' method to search for the substring 'Welcome' in the 'my' string. It finds the first occurrence of 'Welcome' at index 14, and therefore the value of 'i' will be '14'.

3. 'print(my[i:])': This line uses slicing to extract the portion of the string 'my' starting from index 'i' (which is '14') until the end of the string. Slicing is done using the syntax 'string[start:]', where 'start' is the starting index, and ':' indicates that we want to include all characters from the starting index to the end of the string. So, it will print the substring starting from index 14, which is 'Welcome Back Amr'.

So, the output of the code will be:

```
Welcome Back Amr
```

The code locates the index of the substring 'Welcome' in the original string and then extracts the rest of the string starting from that index using slicing.

Q96. In this code snippet, you have two strings, 'my1' and 'my2'. You are using the '.replace()' method to replace all occurrences of the substring 'Amr' in 'my1' with the new substring 'Adel'. Let's go through the code step by step:

```
python
my1 = 'Hello Amr and Welcome Back Amr'
```

```
my2 = my1.replace('Amr', 'Adel')
print(my1)
print(my2)
```

1. `'my1 = 'Hello Amr and Welcome Back Amr''`: This assigns the string `'Hello Amr and Welcome Back Amr'` to the variable `'my1'`.

2. `'my2 = my1.replace('Amr', 'Adel')'`: This line uses the `'.replace()'` method on `'my1'` to replace all occurrences of the substring `'Amr'` with the new substring `'Adel'`. The result is stored in the variable `'my2'`, but it doesn't modify the original string `'my1'`.

3. `'print(my1)'`: This prints the value of `'my1'`. Since `'my1'` remains unchanged after the replacement, it will still be `'Hello Amr and Welcome Back Amr'`.

4. `'print(my2)'`: This prints the value of `'my2'`, which contains the result of the replacement. All occurrences of `'Amr'` in `'my1'` have been replaced with `'Adel'`, so the output will be:

```
Hello Adel and Welcome Back Adel
```

As you can see, the `'.replace()'` method creates a new string with the specified replacements and leaves the original string (`'my1'` in this case) unchanged. The modified string with replacements is stored in the new variable `'my2'`.

Q97. The code you provided is an if statement in Python. It checks if the variable `'x'` is equal to `'5'`, and if that condition is true, it will execute the indented block of code following the `'if'` statement. If the condition is not met (if `'x'` is not equal to `'5'`), the indented block will not be executed.

Here's the code snippet:

```
python
if x == 5:
    print('OK')
```

Explanation:

1. 'x' is a variable that is assumed to be previously defined in your code. Its value will be compared to '5'.
2. 'x == 5': This is the condition that checks if 'x' is equal to '5'. If this condition is true, the code inside the 'if' block will be executed.
3. 'print('OK')': If the condition 'x == 5' is true, the 'print()' function will be called, and it will display the string 'OK' on the screen.

Please note that if 'x' is not defined before this code snippet or if 'x' has a value other than '5', nothing will be printed because the 'print('OK')' statement will not be executed.

Q98. In this code snippet, you have defined the variable 'x' with the value '7'. Then, you have an if statement that checks if 'x' is greater than '5'. If this condition is true, the indented block of code following the 'if' statement will be executed. Let's go through the code step by step:

```
python
x = 7
if x > 5:
    print('OK1')
    print('OK2')
    print('OK3')
```

Explanation:

1. `'x = 7'`: This assigns the value '7' to the variable 'x'.
2. `'if x > 5:'`: This is the condition that checks if the value of 'x' is greater than '5'. Since 'x' has the value '7', the condition is true, and the code inside the 'if' block will be executed.
3. `'print('OK1')'`: When the condition is true, the 'print()' function will be called, and it will display the string 'OK1' on the screen.
4. `'print('OK2')'`: This line will also be executed, and it will display the string 'OK2' on the screen.
5. `'print('OK3')'`: Similarly, this line will be executed, and it will display the string 'OK3' on the screen.

So, the output of the code will be:

```
OK1
OK2
OK3
```

All three strings, 'OK1', 'OK2', and 'OK3', will be printed because the condition `'x > 5'` is true when 'x' has the value '7'.

Q99. In this code snippet, you have defined the variable 'x' with the value '7'. Then, you have an if statement that checks if 'x' is less than or equal to '7'. If this condition is true, the indented block of code following the 'if' statement will be executed. Let's go through the code step by step:

```
python
x = 7
if x <= 7:
    print('OK1')
    print('OK2')
```

```
print('OK3')
```

Explanation:

1. `'x = 7':` This assigns the value '7' to the variable 'x'.
2. `'if x <= 7:':` This is the condition that checks if the value of 'x' is less than or equal to '7'. Since 'x' has the value '7', the condition is true, and the code inside the 'if' block will be executed.
3. `'print('OK1')':` When the condition is true, the 'print()' function will be called, and it will display the string 'OK1' on the screen.
4. `'print('OK2')':` This line will also be executed, and it will display the string 'OK2' on the screen.
5. `'print('OK3')':` Similarly, this line will be executed, and it will display the string 'OK3' on the screen.

So, the output of the code will be:

```
OK1
OK2
OK3
```

All three strings, 'OK1', 'OK2', and 'OK3', will be printed because the condition `'x <= 7'` is true when 'x' has the value '7'. The `'<='` operator checks for "less than or equal to" in this case.

Q100. This code snippet is a simple Python program that takes user input, checks whether the entered number is positive or negative, and prints the corresponding message accordingly. Let's go through the code step by step:

```
python
num = int(input('Enter any number:'))
```

Explanation:

1. `'input('Enter any number:')'`: The `'input()'` function is used to take user input from the console. In this case, it displays the message `'Enter any number:'`, and the user is expected to enter a number.
2. `'int()'`: The `'int()'` function is used to convert the user input, which is by default a string, into an integer data type so that numerical comparisons can be performed.
3. `'num = int(input('Enter any number:'))'`: This line combines the user input and integer conversion. The user-entered number is stored in the variable `'num'`.

Next, there is an `'if-else'` statement:

```
python
if num < 0:
    print('Negative Number')
else:
    print('Positive Number')
```

Explanation:

The `'if'` statement checks whether the value stored in the variable `'num'` is less than `'0'`. If this condition is true, the code inside the `'if'` block will be executed. Otherwise, if the condition is false (meaning the number is greater than or equal to `'0'`), the code inside the `'else'` block will be executed.

- If the user enters a negative number (e.g., `'-5'`), the `'if'` condition `'num < 0'` will be true, and the message `'Negative Number'` will be printed.

- If the user enters a non-negative number (e.g., '0' or '10'), the 'if' condition 'num < 0' will be false, and the message 'Positive Number' will be printed.

So, based on the user input, the program will display either 'Negative Number' or 'Positive Number' as the output.

Q101. This code snippet is a Python program that takes the input of a student's degree (or score), evaluates it based on specific ranges, and then prints the corresponding letter grade. The code uses an 'if-elif-else' statement for conditional branching. Let's go through the code step by step:

```
python
degree = int(input('Enter student degree:'))
```

Explanation:

1. 'input('Enter student degree:')': The 'input()' function is used to take user input from the console. In this case, it displays the message 'Enter student degree:', and the user is expected to enter a numerical degree or score.

2. 'int()': The 'int()' function is used to convert the user input, which is by default a string, into an integer data type so that numerical comparisons can be performed.

3. 'degree = int(input('Enter student degree:'))': This line combines the user input and integer conversion. The user-entered degree or score is stored in the variable 'degree'.

Next, there is an 'if-elif-else' statement that checks the value of 'degree' and assigns a corresponding letter grade:

```
python
if degree < 0 or degree > 100:
    print('Degree Error')
```

```
elif degree < 50:
    print('F')
elif degree < 60:
    print('E')
elif degree < 70:
    print('D')
elif degree < 80:
    print('C')
elif degree < 90:
    print('B')
else:
    print('A')
```

Explanation:

The code goes through the following conditions in order:

1. 'if degree < 0 or degree > 100:': This condition checks if the entered degree is less than '0' or greater than '100'. If the condition is true (i.e., the degree is outside the valid range), it prints the message 'Degree Error'.
2. 'elif degree < 50:': If the degree is not outside the valid range, it checks if it is less than '50'. If true, it prints 'F'.
3. 'elif degree < 60:': If the degree is not less than '50', it checks if it is less than '60'. If true, it prints 'E'.
4. 'elif degree < 70:': And so on, it continues to check if the degree falls into the range of '60' to '69', printing 'D' in that case.
5. 'elif degree < 80:': It checks if the degree falls into the range of '70' to '79', printing 'C' in that case.
6. 'elif degree < 90:': It checks if the degree falls into the range of '80' to '89', printing 'B' in that case.

7. 'else:': If the degree is not within any of the previous ranges, it means it is '90' or above, and it prints 'A'.

So, based on the user's input, the program will print the corresponding letter grade or display 'Degree Error' if the entered degree is outside the range of '0' to '100'.

Q102. In this code snippet, the user is prompted to enter two numbers, 'num1' and 'num2'. The program then compares the two numbers and prints the label of the bigger number. The code uses a conditional expression (ternary operator) to achieve this in a single line. Let's go through the code step by step:

```
python
num1 = int(input('Enter Number 1:'))
num2 = int(input('Enter Number 2:'))
```

Explanation:

1. 'input('Enter Number 1:')': The 'input()' function is used to take user input from the console. In this case, it displays the message 'Enter Number 1:', and the user is expected to enter a number.

2. 'int()': The 'int()' function is used to convert the user input, which is by default a string, into an integer data type so that numerical comparisons can be performed.

3. 'num1 = int(input('Enter Number 1:'))': This line combines the user input and integer conversion. The user-entered number is stored in the variable 'num1'.

4. Similarly, the second number is obtained and stored in the variable 'num2'.

Next, there is a conditional expression (ternary operator):

```
python
```

```
str_big = 'Number 1' if num1 > num2 else 'Number 2'
```

Explanation:

This line uses a conditional expression in the form 'value_if_true if condition else value_if_false'. It checks if 'num1' is greater than 'num2'. If this condition is true, it assigns the string 'Number 1' to the variable 'str_big'. Otherwise, if the condition is false (meaning 'num1' is not greater than 'num2'), it assigns the string 'Number 2' to 'str_big'.

Finally, the bigger number's label is printed:

```
python
print(str_big)
```

Explanation:

The 'print()' function is used to display the value of the variable 'str_big'. This will be the label of the bigger number, either 'Number 1' or 'Number 2', depending on the comparison.

So, based on the user's input, the program will determine which number is bigger and print the corresponding label.

Q103. In this code snippet, you have a 'for' loop that iterates through a tuple containing the values '(1, 2, 3, 4, 5)'. During each iteration, the loop variable 'x' takes on one of the values in the tuple, and the 'print()' function is called to display the value of 'x'. Let's go through the code step by step:

```
python
for x in (1, 2, 3, 4, 5):
    print(x)
```

Explanation:

1. `'(1, 2, 3, 4, 5)'`: This is a tuple containing the values '1', '2', '3', '4', and '5'.
2. `'for x in (1, 2, 3, 4, 5):'`: This is a 'for' loop that iterates over the values in the tuple. During each iteration, the loop variable 'x' takes on one of the values in the tuple '(1, 2, 3, 4, 5)'.
3. `'print(x)'`: During each iteration of the loop, the 'print()' function is called to display the current value of 'x'.

When the code is executed, the output will be:

```
1
2
3
4
5
```

The loop runs five times, with 'x' taking on the values '1', '2', '3', '4', and '5' in each iteration. The 'print()' function displays each value of 'x' on a new line.

Q104. In this code snippet, you have a 'for' loop that iterates through a list containing the values '[10, 20, 30, 40, 50]'. During each iteration, the loop variable 'x' takes on one of the values in the list, and the 'print()' function is called to display the value of 'x'. Let's go through the code step by step:

```
python
for x in [10, 20, 30, 40, 50]:
    print(x)
```

Explanation:

1. `'[10, 20, 30, 40, 50]'`: This is a list containing the values '10', '20', '30', '40', and '50'.
2. `'for x in [10, 20, 30, 40, 50]:'`: This is a 'for' loop that iterates over the values in the list. During each iteration, the loop variable 'x' takes on one of the values in the list '[10, 20, 30, 40, 50]'.
3. `'print(x)'`: During each iteration of the loop, the `'print()'` function is called to display the current value of 'x'.

When the code is executed, the output will be:

```
10
20
30
40
50
```

The loop runs five times, with 'x' taking on the values '10', '20', '30', '40', and '50' in each iteration. The `'print()'` function displays each value of 'x' on a new line.

Q105. In this code snippet, you have a 'for' loop that uses the `'range()'` function to generate a sequence of numbers from '2' up to (but not including) '11', with a step size of '2'. During each iteration of the loop, the loop variable 'x' takes on one of the values in this sequence, and the `'print()'` function is called to display the value of 'x'. Let's go through the code step by step:

```
python
for x in range(2, 11, 2):
    print(x)
```

Explanation:

1. `'range(2, 11, 2)'`: This is a range object that generates a sequence of numbers starting from '2', up to (but not including) '11', with a step size of '2'. It generates the numbers '2', '4', '6', '8', and '10'.

2. `'for x in range(2, 11, 2):'`: This is a 'for' loop that iterates over the numbers generated by the `'range()'` function. During each iteration, the loop variable 'x' takes on one of the values in the sequence.

3. `'print(x)'`: During each iteration of the loop, the `'print()'` function is called to display the current value of 'x'.

When the code is executed, the output will be:

```
2
4
6
8
10
```

The loop runs five times, with 'x' taking on the values '2', '4', '6', '8', and '10' in each iteration. The `'print()'` function displays each value of 'x' on a new line. The `'range()'` function with a step size of '2' creates a sequence of even numbers between '2' and '10' (inclusive).

Q106. In this code snippet, you have a 'for' loop that uses the `'range()'` function to generate a sequence of numbers from '1' up to (but not including) '6'. During each iteration of the loop, the loop variable 'x' takes on one of the values in this sequence. Inside the loop, there is an 'if' statement that checks if 'x' is not equal to '4', and if that condition is true, it executes the `'print()'`

function to display the value of 'x'. Let's go through the code step by step:

```
python
for x in range(1, 6):
    if x != 4:
        print(x)
```

Explanation:

1. 'range(1, 6)': This is a range object that generates a sequence of numbers starting from '1', up to (but not including) '6'. It generates the numbers '1', '2', '3', '4', and '5'.
2. 'for x in range(1, 6)': This is a 'for' loop that iterates over the numbers generated by the 'range()' function. During each iteration, the loop variable 'x' takes on one of the values in the sequence.
3. 'if x != 4:': This 'if' statement checks if the current value of 'x' is not equal to '4'.
4. 'print(x)': If the condition in the 'if' statement is true (i.e., 'x' is not equal to '4'), the 'print()' function is called to display the current value of 'x'.

When the code is executed, the output will be:

```
1
2
3
5
```

The loop runs five times, with 'x' taking on the values '1', '2', '3', '4', and '5' in each iteration. However, when 'x' is equal to '4', the 'if' statement condition 'x != 4' evaluates to 'False', and

therefore, the `'print(x)'` statement is skipped for that particular iteration. As a result, the value `'4'` is not printed, and the output displays only `'1'`, `'2'`, `'3'`, and `'5'`.

Q107. In this code snippet, you have a `'for'` loop that uses the `'range()'` function to generate a sequence of numbers starting from `'5'` and counting down to (but not including) `'0'`. The step size is `'-1'`, meaning it will decrement by `'1'` at each iteration. During each iteration of the loop, the loop variable `'x'` takes on one of the values in this sequence, and the `'print()'` function is called to display the value of `'x'`. Let's go through the code step by step:

```
python
for x in range(5, 0, -1):
    print(x)
```

Explanation:

1. `'range(5, 0, -1)'`: This is a range object that generates a sequence of numbers starting from `'5'` (inclusive) and counting down to (but not including) `'0'`. It generates the numbers `'5'`, `'4'`, `'3'`, `'2'`, and `'1'`.

2. `'for x in range(5, 0, -1):'`: This is a `'for'` loop that iterates over the numbers generated by the `'range()'` function. The step size `'-1'` ensures that the loop variable `'x'` decrements by `'1'` at each iteration.

3. `'print(x)'`: During each iteration of the loop, the `'print()'` function is called to display the current value of `'x'`.

When the code is executed, the output will be:

```
5
4
3
```

```
2
1
```

The loop runs five times, with 'x' taking on the values '5', '4', '3', '2', and '1' in each iteration. The 'print()' function displays each value of 'x' on a new line, starting from '5' and counting down to '1'.

Q108. In this code snippet, you have a 'for' loop that iterates over a range of character codes from the ASCII values of 'A' to 'Z' (inclusive). For each character code, the loop converts it back to its corresponding character using the 'chr()' function and appends it to the 'alpha' string. Additionally, a comma and space ', ' are appended after each character, except for the last one.

Let's go through the code step by step:

```
python
alpha = ''
for x in range(ord('A'), ord('Z')+1):
    alpha += chr(x)
    if x < ord('Z'):
        alpha += ', '
    print(alpha)
```

Explanation:

1. 'alpha = '': This initializes an empty string variable called 'alpha', which will be used to store the characters.

2. 'for x in range(ord('A'), ord('Z')+1)': This 'for' loop iterates over the range of character codes from the ASCII value of 'A' to the ASCII value of 'Z' (inclusive). In ASCII, 'A' has the value '65' and 'Z' has the value '90'.

3. 'alpha += chr(x)': Inside the loop, 'chr(x)' converts the current

character code 'x' back into its corresponding character (e.g., '65' becomes 'A', '66' becomes 'B', and so on), and this character is appended to the 'alpha' string.

4. 'if x < ord('Z'):: This 'if' statement checks if the current character code 'x' is less than the ASCII value of 'Z' ('90'). It is used to add the comma and space ', ' after each character, except for the last character.

5. 'alpha += ', ': If the condition in the 'if' statement is true, the comma and space ', ' are appended to the 'alpha' string.

6. 'print(alpha)': During each iteration of the loop, the current value of the 'alpha' string is printed. As the loop progresses, each character and its corresponding comma-separated string up to the current character are displayed.

When the code is executed, the output will be:

```
A,  
A, B,  
A, B, C,  
A, B, C, D,  
A, B, C, D, E,  
A, B, C, D, E, F,  
A, B, C, D, E, F, G,  
A, B, C, D, E, F, G, H,  
A, B, C, D, E, F, G, H, I,  
A, B, C, D, E, F, G, H, I, J,  
A, B, C, D, E, F, G, H, I, J, K,  
A, B, C, D, E, F, G, H, I, J, K, L,  
A, B, C, D, E, F, G, H, I, J, K, L, M,  
A, B, C, D, E, F, G, H, I, J, K, L, M, N,  
A, B, C, D, E, F, G, H, I, J, K, L, M, N, O,  
A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P,  
A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q,  
A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R,  
A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S,
```

```
A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T,  
A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U,  
A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V,  
A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W,  
A, B, C, D, E, F, G, H, I, J, K,
```

Q109. In this updated code snippet, you have a 'for' loop that uses the 'range()' function to generate a sequence of numbers starting from the ASCII value of 'Z' and counting down to (but not including) the ASCII value of 'A'. The step size is '-1', meaning it will decrement by '1' at each iteration. During each iteration of the loop, the loop variable 'x' takes on one of the values in this sequence, and the 'print()' function is called to display the value of 'alpha', which is the concatenation of the characters corresponding to the ASCII values.

Let's go through the code step by step:

```
python  
alpha = ''  
for x in range(ord('Z'), ord('A')-1, -1):  
    alpha += chr(x)  
    if x > ord('A'):  
        alpha += ', '  
    print(alpha)
```

Explanation:

1. 'alpha = '': This initializes an empty string variable called 'alpha', which will be used to store the characters.
2. 'for x in range(ord('Z'), ord('A')-1, -1)': This 'for' loop iterates over the range of character codes from the ASCII value of 'Z' to the ASCII value of 'A' (inclusive). In ASCII, 'Z' has the value '90', and 'A' has the value '65'.
3. 'alpha += chr(x)': Inside the loop, 'chr(x)' converts the current

character code 'x' back into its corresponding character (e.g., '90' becomes 'Z', '89' becomes 'Y', and so on), and this character is appended to the 'alpha' string.

4. 'if x > ord('A'):: This 'if' statement checks if the current character code 'x' is greater than the ASCII value of 'A' ('65'). It is used to add the comma and space ', ' after each character, except for the first character.

5. 'alpha += ', ': If the condition in the 'if' statement is true, the comma and space ', ' are appended to the 'alpha' string.

6. 'print(alpha)': During each iteration of the loop, the current value of the 'alpha' string is printed. As the loop progresses, each character and its corresponding comma-separated string up to the first character (i.e., 'Z') are displayed.

When the code is executed, the output will be:

```
Z, Y
Z, Y, X
Z, Y, X, W
Z, Y, X, W, V
Z, Y, X, W, V, U
Z, Y, X, W, V, U, T
Z, Y, X, W, V, U, T, S
Z, Y, X, W, V, U, T, S, R
Z, Y, X, W, V, U, T, S, R, Q
Z, Y, X, W, V, U, T, S, R, Q, P
Z, Y, X, W, V, U, T, S, R, Q, P, O
Z, Y, X, W, V, U, T, S, R, Q, P, O, N
Z, Y, X, W, V, U, T, S, R, Q, P, O, N, M
Z, Y, X, W, V, U, T, S, R, Q, P, O, N, M, L
Z, Y, X, W, V, U, T, S, R, Q, P, O, N, M, L, K
Z, Y, X, W, V, U, T, S, R, Q, P, O, N, M, L, K, J
Z, Y, X, W, V, U, T, S, R, Q, P, O, N, M, L, K, J, I
Z, Y, X, W, V, U, T, S, R, Q, P, O, N, M, L, K, J, I, H
Z, Y, X, W, V, U, T, S, R, Q, P, O, N, M, L, K, J, I, H, G
```

```
Z, Y, X, W, V, U, T, S, R, Q, P, O, N, M, L, K, J, I, H, G, F
Z, Y, X, W, V, U, T, S, R, Q, P, O, N, M, L, K, J, I, H, G, F, E
Z, Y, X, W, V, U, T, S, R, Q, P, O, N, M, L, K, J, I, H, G, F, E, D
Z, Y, X, W, V, U, T, S, R, Q, P, O, N, M, L, K, J, I, H, G, F, E, D,
C
Z, Y, X, W, V, U, T, S, R, Q, P, O, N, M, L, K, J, I, H, G, F, E, D,
C, B
Z, Y, X, W, V, U, T, S, R, Q, P, O, N, M, L, K, J, I, H, G, F, E, D,
C, B, A
```

The loop runs twenty-six times, with 'x' taking on the character codes of the uppercase English alphabet in reverse order, and the 'print()' function displays the current value of the 'alpha' string in each iteration. The output shows each character and its corresponding comma-separated string up to the first character 'Z'.

Q110. It seems there is a minor typo in your code snippet. The list of names should be named 'names' instead of '['Ahmed', 'Adel', 'Amr', 'Omar', 'Ali']'. Assuming you are using the correct list of names, the code will work correctly. Let's go through the code step by step:

```
python
names = ['Ahmed', 'Adel', 'Amr', 'Omar', 'Ali']
for x in range(len(names)):
    print('Hello ' + names[x])
```

Explanation:

1. 'names = ['Ahmed', 'Adel', 'Amr', 'Omar', 'Ali']': This line creates a list called 'names' containing five elements, which are strings representing different names.

2. 'for x in range(len(names))': This 'for' loop iterates over the indices of the 'names' list using the 'range()' function with 'len(names)' as the parameter. 'len(names)' returns the length of the 'names' list, which is '5'. The 'range()' function generates a

sequence of numbers from '0' up to (but not including) '5', which corresponds to the indices of the list 'names'.

3. `'print('Hello ' + names[x])'`: During each iteration of the loop, the current name from the 'names' list is accessed using the loop variable 'x'. The `'print()'` function is called to display the greeting `'Hello '` concatenated with the current name.

When the code is executed, the output will be:

```
Hello Ahmed
Hello Adel
Hello Amr
Hello Omar
Hello Ali
```

The loop runs five times, with 'x' taking on the values '0', '1', '2', '3', and '4' in each iteration. The `'print()'` function displays the greeting `'Hello '` followed by the name corresponding to the current index in the 'names' list.

Q111. In this code snippet, you have a list called 'names' containing five elements, each representing a different name. You are using a 'for' loop to iterate over the elements of the 'names' list. During each iteration, the loop variable 'name' takes on one of the names in the list, and the `'print()'` function is called to display the greeting `'Hello '` concatenated with the current name. Let's go through the code step by step:

```
python
names = ['Ahmed', 'Adel', 'Amr', 'Omar', 'Ali']
for name in names:
    print('Hello ' + name)
```

Explanation:

1. `'names = ['Ahmed', 'Adel', 'Amr', 'Omar', 'Ali']'`: This line creates a list called 'names' containing five elements, which are strings representing different names.

2. `'for name in names:'`: This 'for' loop iterates over the elements of the 'names' list. During each iteration, the loop variable 'name' takes on one of the names in the list.

3. `'print('Hello ' + name)'`: During each iteration of the loop, the 'print()' function is called to display the greeting 'Hello ' concatenated with the current 'name'.

When the code is executed, the output will be:

```
Hello Ahmed
Hello Adel
Hello Amr
Hello Omar
Hello Ali
```

The loop runs five times, with 'name' taking on the values 'Ahmed', 'Adel', 'Amr', 'Omar', and 'Ali' in each iteration. The 'print()' function displays the greeting 'Hello ' followed by the current name stored in the variable 'name'. The loop iterates through all the names in the 'names' list, printing a greeting for each name.

Q112. In this code snippet, you have a list called 'my_list' containing four elements of different data types: an integer, a string, a boolean, and a float. You are using a 'for' loop to iterate over the elements of the 'my_list'. During each iteration, the loop variable 'v' takes on one of the elements in the list, and the 'print()' function is called to display the value of 'v' and its corresponding data type using the 'type()' function. Let's go through the code step by step:

```
python
my_list = [3, 'A', True, 5.7]
for v in my_list:
    print(v, type(v))
```

Explanation:

1. `'my_list = [3, 'A', True, 5.7]'`: This line creates a list called `'my_list'` containing four elements. The elements are `'3'` (an integer), `'A'` (a string), `'True'` (a boolean value), and `'5.7'` (a float).

2. `'for v in my_list:'`: This `'for'` loop iterates over the elements of the `'my_list'`. During each iteration, the loop variable `'v'` takes on one of the elements in the list.

3. `'print(v, type(v))'`: During each iteration of the loop, the `'print()'` function is called to display the value of `'v'` and its corresponding data type using the `'type()'` function.

When the code is executed, the output will be:

```
3 <class 'int'>
A <class 'str'>
True <class 'bool'>
5.7 <class 'float'>
```

The loop runs four times, with `'v'` taking on the values `'3'`, `'A'`, `'True'`, and `'5.7'` in each iteration. The `'print()'` function displays the value of `'v'` and its data type obtained using the `'type()'` function. The output shows the value of each element in the list followed by its respective data type.

Q113. In this code snippet, you have a list called `'my_list'` containing four elements of different data types: an integer, a

string, a boolean, and a float. You are using a 'for' loop to iterate over the elements of the 'my_list'. During each iteration, the loop variable 'v' takes on one of the elements in the list, and the 'print()' function is called to display the value of 'v' and its corresponding data type using the 'type()' function. Let's go through the code step by step:

```
python
my_list = [3, 'A', True, 5.7]
for v in my_list:
    print(v, type(v))
```

Explanation:

1. 'my_list = [3, 'A', True, 5.7]': This line creates a list called 'my_list' containing four elements. The elements are '3' (an integer), 'A' (a string), 'True' (a boolean value), and '5.7' (a float).
2. 'for v in my_list:': This 'for' loop iterates over the elements of the 'my_list'. During each iteration, the loop variable 'v' takes on one of the elements in the list.
3. 'print(v, type(v))': During each iteration of the loop, the 'print()' function is called to display the value of 'v' and its corresponding data type using the 'type()' function.

When the code is executed, the output will be:

```
3 <class 'int'>
A <class 'str'>
True <class 'bool'>
5.7 <class 'float'>
```

The loop runs four times, with 'v' taking on the values '3', 'A', 'True', and '5.7' in each iteration. The 'print()' function displays

the value of 'v' and its data type obtained using the 'type()' function. The output shows the value of each element in the list followed by its respective data type.

Q114. In this code snippet, you have a dictionary named 'emp' containing three key-value pairs. The dictionary represents information about an employee, with keys such as 'name', 'city', and 'salary', and their respective values are 'Adel', 'Giza', and '3000'. You are using a 'for' loop to iterate over the keys of the 'emp' dictionary. During each iteration, the loop variable 'x' takes on one of the keys in the dictionary. The 'print()' function is then called to display the corresponding value associated with the current key 'x' in the dictionary.

Let's go through the code step by step:

```
python
emp = {'name': 'Adel', 'city': 'Giza', 'salary': 3000}
for x in emp:
    print(emp[x])
```

Explanation:

1. 'emp = {'name': 'Adel', 'city': 'Giza', 'salary': 3000}': This line creates a dictionary called 'emp' with three key-value pairs. The keys are 'name', 'city', and 'salary', and their respective values are 'Adel', 'Giza', and '3000'.
2. 'for x in emp:': This 'for' loop iterates over the keys of the 'emp' dictionary. During each iteration, the loop variable 'x' takes on one of the keys in the dictionary.
3. 'print(emp[x])': During each iteration of the loop, the 'print()' function is called to display the value associated with the current key 'x' in the 'emp' dictionary. To access the value associated with a specific key in a dictionary, you use square brackets '[]' with the key inside them.

When the code is executed, the output will be:

```
Adel
Giza
3000
```

The loop runs three times, with 'x' taking on the values 'name', 'city', and 'salary' in each iteration. The 'print()' function displays the corresponding value for each key in the 'emp' dictionary. The output shows the values 'Adel', 'Giza', and '3000', which are associated with the keys 'name', 'city', and 'salary', respectively.

Q115. In this code snippet, you have a dictionary named 'emp' containing three key-value pairs. The dictionary represents information about an employee, with keys such as 'name', 'city', and 'salary', and their respective values are 'Ahmed', 'Giza', and '3000'. You are using a 'for' loop to iterate over the items of the 'emp' dictionary. During each iteration, two variables 'k' and 'v' are used to unpack the key-value pairs from the dictionary. The 'print()' function is then called to display the key followed by the corresponding value.

Let's go through the code step by step:

```
python
emp = {'name': 'Ahmed', 'city': 'Giza', 'salary': 3000}
for k, v in emp.items():
    print(str(k) + ': ', v)
```

Explanation:

1. 'emp = {'name': 'Ahmed', 'city': 'Giza', 'salary': 3000}': This line creates a dictionary called 'emp' with three key-value pairs.

The keys are 'name', 'city', and 'salary', and their respective values are 'Ahmed', 'Giza', and '3000'.

2. `'for k, v in emp.items()'`: This 'for' loop iterates over the items (key-value pairs) of the 'emp' dictionary. The 'items()' method returns a view object that contains tuples of key-value pairs.

3. `'print(str(k) + ':', v)'`: During each iteration of the loop, the 'print()' function is called to display the key and its corresponding value. The 'str(k)' converts the key 'k' to a string to ensure proper concatenation.

When the code is executed, the output will be:

```
name: Ahmed
city: Giza
salary: 3000
```

The loop runs three times, with 'k' taking on the values 'name', 'city', and 'salary', and 'v' taking on the corresponding values 'Ahmed', 'Giza', and '3000' in each iteration. The 'print()' function displays each key followed by its corresponding value. The output shows the key-value pairs of the 'emp' dictionary, with each key-value pair displayed on a separate line.

Q116. It seems there is a minor issue with the indentation in your code snippet. The second 'for' loop is not correctly nested within the first one. To achieve the desired output, you need to nest the second 'for' loop inside the first one. Additionally, you should use the loop variable 'x' to access the lists 'family1', 'family2', and 'family3' within 'home1'. Here's the corrected code:

```
python
family1 = ['Ahmed', 'Adel', 'Amr']
family2 = ['Ehab', 'Mahmoud', 'Ezz']
family3 = ['Sarah', 'Hajer', 'Rehab']
```

```
home1 = [family1, family2, family3]

for x in range(len(home1)):
    print('Family:', x + 1)
    for y in range(len(home1[x])):
        print(' Name', y + 1, 'is:', home1[x][y])
```

Explanation:

1. 'family1', 'family2', and 'family3': These are three lists containing the names of family members.
2. 'home1': This is a list containing the three family lists, creating a list of lists.
3. 'for x in range(len(home1))': This 'for' loop iterates over the indices of the 'home1' list. The 'len(home1)' function returns '3', so the loop will run three times, with 'x' taking on the values '0', '1', and '2'.
4. 'print('Family:', x + 1)': During each iteration of the first loop, this line prints the family number (1-based index) using the loop variable 'x'.
5. 'for y in range(len(home1[x]))': This 'for' loop is nested within the first loop and iterates over the indices of the family lists ('family1', 'family2', or 'family3') based on the current index 'x'. It will run three times for each family list.
6. 'print(' Name', y + 1, 'is:', home1[x][y])': During each iteration of the second loop, this line prints the name of each family member, using the loop variable 'y' to access the names within the family list based on the current index 'x'.

When the code is executed, the output will be:

```
Family: 1
```

```
Name 1 is: Ahmed
Name 2 is: Adel
Name 3 is: Amr
Family: 2
Name 1 is: Ehab
Name 2 is: Mahmoud
Name 3 is: Ezz
Family: 3
Name 1 is: Sarah
Name 2 is: Hajer
Name 3 is: Rehab
```

The loop prints the names of family members for each family in 'home1', correctly grouping the names under each family label.

Q117. In your code, you are trying to print the names of family members for each home in 'homes'. However, there is an issue with the nesting of the 'for' loops. The second and third 'for' loops should be nested inside the first one to ensure correct iteration over the homes, families, and names.

Here's the corrected version of the code:

```
python
family1 = ['Adel', 'Amr']
family2 = ['Ehab', 'Ezz']
family3 = ['Sarah', 'Hajer']
family4 = ['Ezzat', 'Foad']
family5 = ['Abdelrahman', 'Abdelkareem']
family6 = ['Ali', 'Ak1']

home1 = [family1, family2, family3]
home2 = [family4, family5, family6]

homes = [home1, home2]
for x in range(len(homes)):
    print('Home', x + 1)
```

```
for i in range(len(homes[x])):
    print(' Family', i + 1)
    for y in range(len(homes[x][i])):
        print(' Name', y + 1, 'is:', homes[x][i][y])
```

Explanation:

1. 'family1', 'family2', ..., 'family6': These are six lists containing the names of family members for each family.
2. 'home1', 'home2': These are two lists, each containing three family lists, creating lists of lists representing homes.
3. 'homes': This is a list containing the two home lists.
4. 'for x in range(len(homes))': This first 'for' loop iterates over the indices of the 'homes' list. The 'len(homes)' function returns '2', so the loop will run twice, with 'x' taking on the values '0' and '1'.
5. 'print('Home', x + 1)': During each iteration of the first loop, this line prints the home number (1-based index) using the loop variable 'x'.
6. 'for i in range(len(homes[x]))': This second 'for' loop is nested within the first loop and iterates over the indices of the family lists ('home1' or 'home2') based on the current index 'x'. It will run three times for each home list.
7. 'print(' Family', i + 1)': During each iteration of the second loop, this line prints the family number (1-based index) using the loop variable 'i'.
8. 'for y in range(len(homes[x][i]))': This third 'for' loop is nested within the second loop and iterates over the indices of the family members in the current family list ('family1', 'family2', etc.), based on the current index 'x' and 'i'.

9. `'print(' Name', y + 1, 'is:', homes[x][i][y])'`: During each iteration of the third loop, this line prints the name of each family member, using the loop variable 'y' to access the names within the family list based on the current index 'x', 'i', and 'y'.

When the code is executed, the output will be:

```
Home 1
  Family 1
    Name 1 is: Adel
    Name 2 is: Amr
  Family 2
    Name 1 is: Ehab
    Name 2 is: Ezz
  Family 3
    Name 1 is: Sarah
    Name 2 is: Hajer
Home 2
  Family 1
    Name 1 is: Ezzat
    Name 2 is: Foaad
  Family 2
    Name 1 is: Abdelrahman
    Name 2 is: Abdelkareem
  Family 3
    Name 1 is: Ali
    Name 2 is: Akl
```

The nested 'for' loops print the names of family members for each home and family in 'homes', correctly grouping the names under each home and family label.

Q118. In this code snippet, you are using the `'enumerate()'` function to iterate over a list of names and print both the index and the corresponding name. The `'enumerate()'` function is used to add a counter to an iterable, such as a list, and return an enumerate

object.

Let's go through the code step by step:

```
python
```

```
for i, name in enumerate(['amr', 'ali', 'ezz']):  
    print(i, name)
```

Explanation:

1. `'enumerate(['amr', 'ali', 'ezz'])'`: The `'enumerate()'` function takes the list `['amr', 'ali', 'ezz']` as an argument and returns an enumerate object. The enumerate object contains tuples, each containing the index and the corresponding element from the list.
2. `'for i, name in enumerate(['amr', 'ali', 'ezz']):'`: This `'for'` loop iterates over the enumerate object. The `'i'` variable represents the index, and the `'name'` variable represents the corresponding name.
3. `'print(i, name)'`: During each iteration of the loop, the `'print()'` function is called to display the index and the name for each element in the list.

When the code is executed, the output will be:

```
0 amr  
1 ali  
2 ezz
```

The loop iterates three times, with `'i'` taking on the values `'0'`, `'1'`, and `'2'`, and `'name'` taking on the corresponding names `'amr'`, `'ali'`, and `'ezz'` in each iteration. The `'print()'` function displays the index and the corresponding name for each element in the list. The output shows the index (starting from `'0'`) and the name for each name in the list.

Q119. In your code snippet, you have three dictionaries named 'person1', 'person2', and 'person3', each representing information about a person's name and salary. You also have a list called 'persons', which contains these three dictionaries. You are using two 'for' loops to iterate over the list of dictionaries and print the information for each person.

Let's go through the code step by step:

```
python
person1 = {'name': 'Amr', 'salary': 5000}
person2 = {'name': 'Ali', 'salary': 4000}
person3 = {'name': 'Ezz', 'salary': 3000}

persons = [person1, person2, person3]

for x in range(len(persons)):
    print('Person', x + 1)

for index, (k, v) in enumerate(persons[x].items()):
    print(' ', index + 1, ': ', k, v)
```

Explanation:

1. 'person1', 'person2', 'person3': These are three dictionaries, each containing the keys 'name' and 'salary' with the respective values.
2. 'persons': This is a list containing the three dictionaries, creating a list of dictionaries representing information about persons.
3. 'for x in range(len(persons))': This first 'for' loop iterates over the indices of the 'persons' list. The 'len(persons)' function returns '3', so the loop will run three times, with 'x' taking on the values '0', '1', and '2'.
4. 'print('Person', x + 1)': During each iteration of the first loop,

this line prints the person number (1-based index) using the loop variable 'x'.

5. `'for index, (k, v) in enumerate(persons[x].items()):'` This second 'for' loop is nested within the first loop and iterates over the items (key-value pairs) of the dictionary corresponding to the current index 'x' in the 'persons' list. It will run twice for each person dictionary.

6. `'print(' ', index + 1, ':', k, v)'`: During each iteration of the second loop, this line prints the index (1-based index), key, and value for each key-value pair in the current person dictionary.

When the code is executed, the output will be:

```
Person 1
Person 2
Person 3
  1 : name Amr
  2 : salary 5000
  1 : name Ali
  2 : salary 4000
  1 : name Ezz
  2 : salary 3000
```

The nested 'for' loops print the information about each person, including their name and salary. The output shows the person number, followed by the key-value pairs (name and salary) for each person in the 'persons' list.

Q120. In this code snippet, you are using a 'while' loop to print the value of the variable 'x' while it is less than or equal to '5'. The 'while' loop continues to run as long as the condition `'x <= 5'` is true. In each iteration, the current value of 'x' is printed, and then 'x' is incremented by '1' using the statement `'x += 1'`, which is equivalent to `'x = x + 1'`.

Let's go through the code step by step:

```
python
x = 1
while x <= 5:
    print(x)
    x += 1
```

Explanation:

1. 'x = 1': This line initializes the variable 'x' with the value '1'.
2. 'while x <= 5:': This 'while' loop runs as long as the condition 'x <= 5' is true. The loop will terminate when 'x' becomes greater than '5'.
3. 'print(x)': During each iteration of the loop, the current value of 'x' is printed.
4. 'x += 1': This statement increments the value of 'x' by '1' after each iteration of the loop. It is equivalent to 'x = x + 1'.

When the code is executed, the output will be:

```
1
2
3
4
5
```

The loop starts with 'x = 1', and in each iteration, the value of 'x' is printed. After each iteration, 'x' is incremented by '1', and the loop continues until 'x' becomes '6', at which point the condition 'x <= 5' becomes 'False', and the loop terminates. The output shows the

values of 'x' from '1' to '5', printed on separate lines.

Q121. In this code snippet, you are using a 'while' loop to print the value of the variable 'x' while it is greater than '0'. The 'while' loop continues to run as long as the condition 'x > 0' is true. In each iteration, the current value of 'x' is printed, and then 'x' is decremented by '1' using the statement 'x -= 1', which is equivalent to 'x = x - 1'.

Let's go through the code step by step:

```
python
x = 5
while x > 0:
    print(x)
    x -= 1
```

Explanation:

1. 'x = 5': This line initializes the variable 'x' with the value '5'.
2. 'while x > 0:': This 'while' loop runs as long as the condition 'x > 0' is true. The loop will terminate when 'x' becomes '0' or less.
3. 'print(x)': During each iteration of the loop, the current value of 'x' is printed.
4. 'x -= 1': This statement decrements the value of 'x' by '1' after each iteration of the loop. It is equivalent to 'x = x - 1'.

When the code is executed, the output will be:

```
5
4
3
```

```
2
1
```

The loop starts with 'x = 5', and in each iteration, the value of 'x' is printed. After each iteration, 'x' is decremented by '1', and the loop continues until 'x' becomes '0', at which point the condition 'x > 0' becomes 'False', and the loop terminates. The output shows the values of 'x' from '5' to '1', printed on separate lines in descending order.

Q122. In this code snippet, you are using a 'while' loop to print the value of the variable 'x' while it is less than or equal to '10'. The 'while' loop continues to run as long as the condition 'x <= 10' is true. In each iteration, the current value of 'x' is printed, and then 'x' is incremented by '2' using the statement 'x += 2', which is equivalent to 'x = x + 2'.

Let's go through the code step by step:

```
python
x = 2
while x <= 10:
    print(x)
    x += 2
```

Explanation:

1. 'x = 2': This line initializes the variable 'x' with the value '2'.
2. 'while x <= 10:': This 'while' loop runs as long as the condition 'x <= 10' is true. The loop will terminate when 'x' becomes greater than '10'.
3. 'print(x)': During each iteration of the loop, the current value of 'x' is printed.

4. `'x += 2'`: This statement increments the value of `'x'` by `'2'` after each iteration of the loop. It is equivalent to `'x = x + 2'`.

When the code is executed, the output will be:

```
2
4
6
8
10
```

The loop starts with `'x = 2'`, and in each iteration, the value of `'x'` is printed. After each iteration, `'x'` is incremented by `'2'`, and the loop continues until `'x'` becomes `'12'`, at which point the condition `'x <= 10'` becomes `'False'`, and the loop terminates. The output shows the even values of `'x'` from `'2'` to `'10'`, printed on separate lines.

Q123. In this code snippet, you are using a `'while'` loop to print the value of the variable `'x'` while it is less than or equal to `'10'`. The `'while'` loop continues to run as long as the condition `'x <= 10'` is true. In each iteration, the current value of `'x'` is printed, and then `'x'` is incremented by `'2'` using the statement `'x += 2'`, which is equivalent to `'x = x + 2'`.

Let's go through the code step by step:

```
python
x = 1
while x <= 10:
    print(x)
    x += 2
```

Explanation:

1. `'x = 1':` This line initializes the variable `'x'` with the value `'1'`.
2. `'while x <= 10:':` This `'while'` loop runs as long as the condition `'x <= 10'` is true. The loop will terminate when `'x'` becomes greater than `'10'`.
3. `'print(x)':` During each iteration of the loop, the current value of `'x'` is printed.
4. `'x += 2':` This statement increments the value of `'x'` by `'2'` after each iteration of the loop. It is equivalent to `'x = x + 2'`.

When the code is executed, the output will be:

```
1
3
5
7
9
```

The loop starts with `'x = 1'`, and in each iteration, the value of `'x'` is printed. After each iteration, `'x'` is incremented by `'2'`, and the loop continues until `'x'` becomes `'11'`, at which point the condition `'x <= 10'` becomes `'False'`, and the loop terminates. The output shows the odd values of `'x'` from `'1'` to `'9'`, printed on separate lines.

Q124. Now, let's see how the code works:

1. Initially, `'x'` is set to `'1'`.
2. The loop starts. The current value of `'x'` is `'1'`, so it satisfies the condition `'x <= 10'`.
3. It prints `'1'`.

4. The line `'x += 2'` increments the value of `'x'` by `'2'`, making it `'3'`.
5. The loop continues with the updated value of `'x'` (which is `'3'` now).
6. It prints `'3'`.
7. The line `'x += 2'` increments `'x'` by `'2'` again, making it `'5'`.
8. The loop continues with the updated value of `'x'` (which is `'5'` now).
9. It prints `'5'`.
10. The line `'x += 2'` increments `'x'` by `'2'` again, making it `'7'`.
11. The loop continues with the updated value of `'x'` (which is `'7'` now).
12. It prints `'7'`.
13. The line `'x += 2'` increments `'x'` by `'2'` again, making it `'9'`.
14. The loop continues with the updated value of `'x'` (which is `'9'` now).
15. It prints `'9'`.
16. The line `'x += 2'` increments `'x'` by `'2'` again, making it `'11'`.
17. Now, the condition `'x <= 10'` is no longer true, so the loop stops executing.
18. The `'else'` block is executed.
19. It prints `'X After Loop Is:'` followed by the final value of `'x'`, which is `'11'`.

The final output will be:

```
1
3
5
7
9
X After Loop Is: 11
```

The loop prints odd numbers from '1' to '9', and after the loop, it prints the final value of 'x', which is '11'.

Q125. In this code snippet, you have a 'while' loop followed by an 'else' block. The 'while' loop runs as long as the condition 'x < 1' is true. However, since the initial value of 'x' is '1', the condition 'x < 1' is initially false, and the 'while' loop is not executed at all. Instead, the code inside the 'else' block will be executed.

Let's go through the code step by step:

```
python
x = 1
while x < 1:
    print(x)
    x += 2
else:
    print('Condition is False')
```

Explanation:

1. 'x = 1': This line initializes the variable 'x' with the value '1'.
2. 'while x < 1:': This 'while' loop runs as long as the condition 'x

`< 1` is true. However, since `'x'` is initially `'1'`, the condition is false from the beginning, and the loop is not executed.

3. `'print('Condition is False')'`: Since the `'while'` loop is not executed due to the false condition, the code inside the `'else'` block is executed. The `'else'` block is associated with the `'while'` loop and is executed when the loop condition becomes false.

When the code is executed, the output will be:

```
Condition is False
```

As explained earlier, the `'while'` loop is not executed because the condition `'x < 1'` is false from the beginning. Therefore, the code inside the `'else'` block is executed, and the message `'Condition is False'` is printed.

Q126. Let's go through the code step by step, explaining each line:

1. `'my_list = [7, 'A', 9.9, False]'`: Here, a list named `'my_list'` is created with four elements: an integer `'7'`, a character `'A'`, a floating-point number `'9.9'`, and a boolean value `'False'`.

2. `'x = 0'`: The variable `'x'` is initialized with the value `'0'`. This variable will be used to keep track of the current index while iterating through `'my_list'`.

3. `'while x < len(my_list):'`: This starts a `'while'` loop that will execute the indented block of code as long as the condition `'x < len(my_list)'` is true. `'len(my_list)'` returns the number of elements in `'my_list'`.

4. `'print(my_list[x], type(my_list[x]))'`: Inside the loop, it prints the element at the current index `'x'` in `'my_list'`, followed by its data type using the `'type()'` function.

5. `'x += 1'`: This line increments the value of `'x'` by `'1'` in each iteration of the loop. This ensures that the loop will move on to the next element in `'my_list'` in the next iteration.

6. The loop continues to iterate through `'my_list'` until `'x'` becomes equal to or greater than the length of `'my_list'`, at which point the loop will terminate.

7. After the loop finishes, the program prints the final output, which shows the elements of `'my_list'` along with their respective data types.

Let's see how the output will be based on the provided `'my_list'`:

```
7 <class 'int'>
A <class 'str'>
9.9 <class 'float'>
False <class 'bool'>
```

The loop prints each element of `'my_list'` and its corresponding data type. The output shows that the first element (`'7'`) is an integer, the second element (`'A'`) is a string, the third element (`'9.9'`) is a float, and the fourth element (`'False'`) is a boolean.

Q127. In this code snippet, you have a dictionary `'emp'` containing information about an employee's name, city, and salary. You are using a `'while'` loop to iterate over the keys of the `'emp'` dictionary and print the corresponding values.

Let's go through the code step by step:

```
python
emp = {'name': 'Adel', 'city': 'Giza', 'salary': 3000}
my_keys = list(emp.keys())
x = 0
while x < len(emp):
```

```
print(emp[my_keys[x]])  
x += 1
```

Explanation:

1. `'emp = {'name': 'Adel', 'city': 'Giza', 'salary': 3000}'`: This line creates a dictionary named 'emp' containing three key-value pairs. The keys are 'name', 'city', and 'salary', and their respective values are 'Adel', 'Giza', and '3000'.

2. `'my_keys = list(emp.keys())'`: This line converts the keys of the 'emp' dictionary into a list called 'my_keys'. This is done so that you can use an index to access the keys in the 'while' loop.

3. `'x = 0'`: This line initializes the variable 'x' with the value '0'. This variable will be used as the loop control variable to access the keys in 'my_keys' list.

4. `'while x < len(emp):'`: This 'while' loop runs as long as the condition 'x < len(emp)' is true. The 'len(emp)' function returns '3' (the number of keys in the 'emp' dictionary), so the loop will run three times, with 'x' taking on the values '0', '1', and '2'.

5. `'print(emp[my_keys[x]])'`: During each iteration of the loop, the current key from the 'my_keys' list is used as an index to access the corresponding value from the 'emp' dictionary, which is then printed.

6. `'x += 1'`: This statement increments the value of 'x' by '1' after each iteration of the loop. It is equivalent to `'x = x + 1'`.

When the code is executed, the output will be:

```
Adel  
Giza  
3000
```

The loop iterates three times, and in each iteration, it accesses the corresponding keys from the 'my_keys' list ('name', 'city', and 'salary') and uses them as indices to access the corresponding values from the 'emp' dictionary ('Adel', 'Giza', and '3000'). The values are then printed on separate lines.

Q128. In this code snippet, you have a 'while' loop with the condition 'True', which means the loop will run indefinitely as long as the condition remains true. The loop will only terminate if you manually stop the execution or encounter a 'break' statement within the loop block.

Let's go through the code step by step:

```
python
x = 1
while True:
    print(x)
    x += 1
```

Explanation:

1. 'x = 1': This line initializes the variable 'x' with the value '1'.
2. 'while True:': This 'while' loop has a condition 'True', which means it will run indefinitely since the condition is always true. This creates an infinite loop.
3. 'print(x)': During each iteration of the loop, the current value of 'x' is printed.
4. 'x += 1': This statement increments the value of 'x' by '1' after each iteration of the loop. It is equivalent to 'x = x + 1'.

When the code is executed, it will continuously print the values of 'x' starting from '1' and increasing by '1' in each iteration. The

loop will continue indefinitely, printing an infinite sequence of numbers until you manually stop the execution.

Please exercise caution when running code with an infinite loop, as it can lead to high CPU usage and may require you to forcibly terminate the program. If you want to stop the loop, you can interrupt the program's execution using keyboard shortcuts (e.g., 'Ctrl + C' in most terminals or 'Kernel > Interrupt' in Jupyter Notebook).

Q129. In this code snippet, you have a 'while' loop that continues until the variable 'again' is equal to 'n'. The loop prompts the user to enter their name, greets them with "Hello," and then asks if they want to repeat the process.

Let's go through the code step by step:

```
python
again = 'y'
while again == 'y':
    name = input('Enter your name:')
    print('Hello ' + name)
    again = input('Again(y/n)?:')
```

Explanation:

1. 'again = 'y'': This line initializes the variable 'again' with the value 'y'.
2. 'while again == 'y':': This 'while' loop runs as long as the condition 'again == 'y'' is true. If the user enters 'y' when asked if they want to repeat, the loop will continue. If the user enters 'n', the loop will terminate.
3. 'name = input('Enter your name:')': This line prompts the user to enter their name. The input provided by the user is stored in the variable 'name'.

4. `'print('Hello ' + name)'`: This line prints the greeting message "Hello" followed by the name entered by the user.

5. `'again = input('Again(y/n)?:')'`: This line prompts the user to enter either `'y'` or `'n'`, asking if they want to repeat the process. The input provided by the user is stored in the variable `'again'`, which determines whether the loop will continue or terminate.

When the code is executed:

- The loop starts with `'again'` equal to `'y'`.
- The user is prompted to enter their name.
- The program greets the user with "Hello" followed by the name.
- The user is prompted again if they want to repeat the process.
- If the user enters `'y'`, the loop starts over and repeats the steps.
- If the user enters `'n'`, the loop terminates, and the program exits.

This code allows the user to input their name, get greeted, and then decide whether they want to repeat the process. The loop will keep running until the user enters `'n'` to stop the repetition.

Q130. In this code snippet, you are using the `'itertools.count()'` function to create an infinite loop that generates an increasing sequence of numbers starting from the specified value (the default is `'0'`).

Let's go through the code step by step:

```
python
from itertools import count

for x in count():
    print(x)
```

Explanation:

1. `'from itertools import count'`: This line imports the `'count'` function from the `'itertools'` module. The `'count'` function generates an infinite arithmetic sequence of numbers.
2. `'for x in count()'`: This `'for'` loop uses the `'count()'` function to generate an infinite sequence of numbers. The loop will run indefinitely since the `'count()'` function continuously generates the next number in the sequence.
3. `'print(x)'`: During each iteration of the loop, the current value of `'x'` (the number in the sequence) is printed.

When the code is executed, it will continuously print an infinite sequence of numbers starting from `'0'` (by default) and increasing by `'1'` in each iteration. The loop will keep running, generating and printing an infinite sequence of numbers until you manually stop the execution.

Again, please exercise caution when running code with an infinite loop, as it can lead to high CPU usage and may require you to forcibly terminate the program. If you want to stop the loop, you can interrupt the program's execution using keyboard shortcuts (e.g., `'Ctrl + C'` in most terminals or `'Kernel > Interrupt'` in Jupyter Notebook).

Q131. In this code snippet, you have a `'for'` loop that iterates over the numbers from `'1'` to `'5'` (exclusive). However, you have a conditional statement with `'break'`, which means the loop will terminate prematurely when the value of `'x'` becomes `'4'`.

Let's go through the code step by step:

```
python
for x in range(1, 6):
    if x == 4:
```



```
        break
    print(x)
```

Explanation:

1. 'for x in range(1, 6)': This 'for' loop iterates over the numbers from '1' to '5' (exclusive). In each iteration, 'x' takes the values '1', '2', '3', and '4'.

2. 'if x == 4:': During each iteration of the loop, this conditional statement checks if 'x' is equal to '4'.

3. 'break': If the condition 'x == 4' is true, the 'break' statement is executed. This immediately terminates the loop, and the program continues executing the code after the loop.

4. 'print(x)': Before the 'break' statement is executed, the current value of 'x' is printed during each iteration of the loop for 'x' equal to '1', '2', and '3'. However, the loop is terminated before 'x' becomes '4'.

When the code is executed, the output will be:

```
1
2
3
```

The loop iterates through the numbers from '1' to '3', and during each iteration, the value of 'x' is printed. However, when 'x' becomes '4', the loop is terminated due to the 'break' statement, and the value '4' is not printed. The output shows the numbers '1', '2', and '3', each printed on separate lines.

Q132. In this code snippet, you have a 'while' loop that iterates over the numbers starting from '1' until 'x' becomes greater than

'5'. Within the loop, you have a conditional statement with a 'break', which means the loop will terminate prematurely when 'x' becomes greater than '5'.

Let's go through the code step by step:

```
python
x = 1
while x <= 100:
    if x > 5:
        break
    print(x)
    x += 1
```

Explanation:

1. 'x = 1': This line initializes the variable 'x' with the value '1'.
2. 'while x <= 100:': This 'while' loop runs as long as the condition 'x <= 100' is true. The loop will continue until 'x' becomes greater than '100'.
3. 'if x > 5:': During each iteration of the loop, this conditional statement checks if 'x' is greater than '5'.
4. 'break': If the condition 'x > 5' is true, the 'break' statement is executed. This immediately terminates the loop, and the program continues executing the code after the loop.
5. 'print(x)': Before the 'break' statement is executed, the current value of 'x' is printed during each iteration of the loop. However, when 'x' becomes greater than '5', the loop is terminated.
6. 'x += 1': This statement increments the value of 'x' by '1' after each iteration of the loop. It is equivalent to 'x = x + 1'.

When the code is executed, the output will be:

```
1
2
3
4
5
```

The loop iterates through the numbers from '1' to '5', and during each iteration, the value of 'x' is printed. However, when 'x' becomes '6', which is greater than '5', the loop is terminated due to the 'break' statement, and the program continues executing the code after the loop. The output shows the numbers '1', '2', '3', '4', and '5', each printed on separate lines.

Q133. In this code snippet, you have a 'while' loop that iterates over the numbers starting from '1' until 'x' becomes greater than '3'. Within the loop, you have a conditional statement with a 'break', which means the loop will terminate prematurely when 'x' becomes greater than '3'.

Let's go through the code step by step:

```
python
x = 1
while x <= 100:
    if x > 3:
        break
    print('OK', x)
    x += 1
```

Explanation:

1. 'x = 1': This line initializes the variable 'x' with the value '1'.

2. `'while x <= 100:'`: This `'while'` loop runs as long as the condition `'x <= 100'` is true. The loop will continue until `'x'` becomes greater than `'100'`.

3. `'if x > 3:'`: During each iteration of the loop, this conditional statement checks if `'x'` is greater than `'3'`.

4. `'break'`: If the condition `'x > 3'` is true, the `'break'` statement is executed. This immediately terminates the loop, and the program continues executing the code after the loop.

5. `'print('OK', x)'`: Before the `'break'` statement is executed, the current value of `'x'` is printed along with the string `'OK'` during each iteration of the loop. However, when `'x'` becomes greater than `'3'`, the loop is terminated.

6. `'x += 1'`: This statement increments the value of `'x'` by `'1'` after each iteration of the loop. It is equivalent to `'x = x + 1'`.

When the code is executed, the output will be:

OK 1

OK 2

OK 3

The loop iterates through the numbers from `'1'` to `'3'`, and during each iteration, the value of `'x'` is printed along with the string `'OK'`. However, when `'x'` becomes `'4'`, which is greater than `'3'`, the loop is terminated due to the `'break'` statement, and the program continues executing the code after the loop. The output shows `'OK 1'`, `'OK 2'`, and `'OK 3'`, each printed on separate lines.

Q134. In this code snippet, you have a list of numbers `'numbers'`, and you want to calculate the sum of all non-zero elements in the list using a `'for'` loop. The loop iterates over the indices of the list, and whenever it encounters a zero, it skips to the next iteration

using the 'continue' statement.

Let's go through the code step by step:

```
python
numbers = [5, 2, 0, 3, 0, 7]
mysum = 0
print('All Is:', len(numbers))
for x in range(len(numbers)):
    if numbers[x] == 0:
        continue
    mysum += numbers[x]
    print('Sum OK Without Zero(s)', 'x:', x)

print('Sum:', mysum)
```

Explanation:

1. 'numbers = [5, 2, 0, 3, 0, 7]': This line creates a list called 'numbers' containing integers.
2. 'mysum = 0': This line initializes the variable 'mysum' with the value '0'. This variable will be used to store the sum of all non-zero elements in the list.
3. 'print('All Is:', len(numbers))': This line prints the length of the 'numbers' list, which is the total number of elements in the list.
4. 'for x in range(len(numbers))': This 'for' loop iterates over the indices of the 'numbers' list using the 'range()' function. The loop runs for each index 'x' from '0' to 'len(numbers)-1'.
5. 'if numbers[x] == 0': During each iteration of the loop, this conditional statement checks if the element at index 'x' in the 'numbers' list is equal to '0'.
6. 'continue': If the condition 'numbers[x] == 0' is true, the

'continue' statement is executed. This means that if the current element is '0', the loop skips to the next iteration without executing the subsequent statements.

7. 'mysum += numbers[x]': If the current element is non-zero (i.e., not equal to '0'), the value of that element is added to the 'mysum' variable.

8. 'print('Sum OK Without Zero(s)', 'x:', x)': This line prints a message along with the current index 'x' during each iteration of the loop for non-zero elements.

9. 'print('Sum:', mysum)': After the loop finishes, this line prints the final value of 'mysum', which represents the sum of all non-zero elements in the list.

When the code is executed, the output will be:

```
All Is: 6
Sum OK Without Zero(s) x: 0
Sum OK Without Zero(s) x: 1
Sum OK Without Zero(s) x: 3
Sum OK Without Zero(s) x: 5
Sum: 17
```

The loop iterates through the 'numbers' list, and for each non-zero element, it adds the element to the 'mysum' variable. The loop skips the elements with the value '0' using the 'continue' statement. The output shows the indices ('x') of the non-zero elements along with the final sum of those elements, which is '17'.

Q135. In this code snippet, you are using list comprehensions to generate two lists: one containing numbers from 0 to 10, and another containing characters (letters) from 'A' to 'Z'.

> Indented block

Let's go through the code step by step:

```
python
numbers = [num for num in range(11)]
print(numbers)
```

Explanation:

1. `'numbers = [num for num in range(11)]'`: This line uses a list comprehension to generate a list named 'numbers' containing numbers from '0' to '10'. The 'range(11)' function generates numbers from '0' to '10', and the list comprehension iterates over each number in that range and adds it to the list.

2. `'print(numbers)'`: This line prints the list 'numbers'.

When the code is executed, the output will be:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

The list 'numbers' contains the numbers from '0' to '10' in ascending order.

Now, let's look at the second part of the code:

```
python
numbers = [chr(num) for num in range(ord('A'), ord('Z') + 1)]
print(numbers)
```

Explanation:

1. `'numbers = [chr(num) for num in range(ord('A'), ord('Z') + 1)]'`:

This line uses a list comprehension to generate a list named 'numbers' containing characters (letters) from 'A' to 'Z'. The 'range(ord('A'), ord('Z') + 1)' function generates a range of Unicode code points corresponding to the characters from 'A' to 'Z'. The 'chr()' function then converts each code point back to its corresponding character, and the list comprehension iterates over each character and adds it to the list.

2. 'print(numbers)': This line prints the list 'numbers'.

When the code is executed, the output will be:

```
['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M',  
'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z']
```

The list 'numbers' contains all the uppercase English letters from 'A' to 'Z' in alphabetical order.

Q136. In this code snippet, you are using a list comprehension to generate a list named 'numbers' that contains even numbers from '0' to '20'.

Let's go through the code step by step:

```
python  
numbers = [num for num in range(21) if num % 2 == 0]  
print(numbers)
```

Explanation:

1. 'numbers = [num for num in range(21) if num % 2 == 0]': This line uses a list comprehension to generate a list named 'numbers'. The list comprehension iterates over the numbers from '0' to '20' (inclusive) using 'range(21)'. For each number 'num', the condition 'num % 2 == 0' checks if the number is even (divisible by 2).

2. `'print(numbers)'`: This line prints the list `'numbers'`.

When the code is executed, the output will be:

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

The list `'numbers'` contains all the even numbers from `'0'` to `'20'` (inclusive). These are the numbers that, when divided by `'2'`, have no remainder. Thus, the output shows all the even numbers in ascending order.

Q137. In this code snippet, you are working with the `'datetime'` module in Python to work with dates and times. Let's go through the code and explain each part:

```
python
import datetime
```

Explanation: This line imports the `'datetime'` module, which allows you to work with dates and times in Python.

```
python
dt1 = datetime.datetime.now()
```

Explanation: This line creates a variable `'dt1'` and assigns it the current date and time using the `'datetime.now()'` function. `'datetime.datetime'` is the class used to represent date and time in Python.

```
python
dt2 = datetime.datetime.now().date()
```

Explanation: This line creates a variable 'dt2' and assigns it the current date by accessing the 'date()' method of the 'datetime' object returned by 'datetime.datetime.now()'.

```
python
dt3 = datetime.datetime.now().time()
```

Explanation: This line creates a variable 'dt3' and assigns it the current time by accessing the 'time()' method of the 'datetime' object returned by 'datetime.datetime.now()'.

```
python
dt4 = datetime.date(2005, 12, 31)
```

Explanation: This line creates a variable 'dt4' and assigns it a specific date, December 31, 2005. The 'datetime.date()' function is used to create a date object with the provided year, month, and day.

```
python
print(dt1)
print(dt2)
print(dt3)
print(dt4)
print(dt5)
```

Explanation: These lines print the values of the variables 'dt1', 'dt2', 'dt3', 'dt4', and 'dt5'. However, there is no 'dt5' variable defined in the code, so attempting to print it will result in a 'NameError'.

When the code is executed, the output will be:

```
2023-07-26 10:45:00.012345 # Example date and time (current date and
time)
2023-07-26 # Example date (current date)
```

```
10:45:00.012345 # Example time (current time)
```

```
2005-12-31 # Specific date
```

```
Traceback (most recent call last):
```

```
File "filename.py", line 13, in <module>
```

```
    print(dt5)
```

```
NameError: name 'dt5' is not defined
```

Note: The actual date and time values in the output will be different as they reflect the current date and time at the time of execution.

Q138. In this code snippet, you are using the 'datetime' module to work with the current date and time and formatting the information into different date and time formats.

Let's go through the code step by step:

```
python
```

```
import datetime
```

Explanation: This line imports the 'datetime' module, which allows you to work with dates and times in Python.

```
python
```

```
now = datetime.datetime.now()
```

Explanation: This line retrieves the current date and time using the 'datetime.now()' method. The variable 'now' holds a 'datetime' object representing the current date and time.

```
python
```

```
d = str(now.day)
```

```
m = str(now.month)
```

```
y = str(now.year)
```

```
h = str(now.hour)
```

```
mi = str(now.minute)
```

```
s = str(now.second)
ms = str(now.microsecond)
```

Explanation: These lines extract individual components (day, month, year, hour, minute, second, and microsecond) from the 'now' 'datetime' object and convert them to strings using the 'str()' function. The reason for converting them to strings is to format them later when printing.

```
python
print(d + '-' + m + '-' + y)
print(y + '/' + m + '/' + d)
print(d + '-' + m + '-' + y + '\t' + h + ':' + mi + ':' + s)
```

Explanation: These lines print the current date and time in different formats:

1. 'print(d + '-' + m + '-' + y)': This line prints the date in the format "day-month-year".
2. 'print(y + '/' + m + '/' + d)': This line prints the date in the format "year/month/day".
3. 'print(d + '-' + m + '-' + y + '\t' + h + ':' + mi + ':' + s)': This line prints the date and time in the format "day-month-year hour:minute:second". The '\t' is used to add a tab character for better formatting between the date and time.

When the code is executed, the output will look like:

```
26-7-2023
2023/7/26
26-7-2023    10:45:0
```

The actual date, time, and formatting in the output will depend on

the current date and time at the moment of execution.

Q139. In this code snippet, you are using the 'datetime' module to work with the current date and time and formatting it into different date and time representations using the 'strftime()' method.

Let's go through the code step by step:

```
python
import datetime
now = datetime.datetime.now()
```

Explanation: These lines import the 'datetime' module and retrieve the current date and time using the 'datetime.now()' method. The variable 'now' holds a 'datetime' object representing the current date and time.

```
python
print('Long day name:', now.strftime('%A'))
print('Short day name:', now.strftime('%a'))
```

Explanation: These lines use the 'strftime()' method to format the date and time:

- '%A': Represents the full name of the day of the week (e.g., Monday, Tuesday, etc.).
- '%a': Represents the abbreviated name of the day of the week (e.g., Mon, Tue, etc.).

```
python
print('Long month name:', now.strftime('%B'))
print('Short month name:', now.strftime('%b'))
```

Explanation: These lines use the 'strftime()' method to format the date and time:

- '%B': Represents the full name of the month (e.g., January, February, etc.).
- '%b': Represents the abbreviated name of the month (e.g., Jan, Feb, etc.).

```
python
print('Date time:', now.strftime('%c'))
print('Day of month:', now.strftime('%d'))
print('Hour number 24:', now.strftime('%H'))
print('Hour number 12:', now.strftime('%I'))
```

Explanation: These lines use the 'strftime()' method to format the date and time:

- '%c': Represents the date and time using the locale's format (e.g., Tue Jul 26 10:45:00 2023).
- '%d': Represents the day of the month as a zero-padded number (e.g., 01, 02, ..., 31).
- '%H': Represents the hour (24-hour clock) as a zero-padded number (e.g., 00, 01, ..., 23).
- '%I': Represents the hour (12-hour clock) as a zero-padded number (e.g., 01, 02, ..., 12).

```
python
print('Day of year:', now.strftime('%j'))
print('Month of year:', now.strftime('%m'))
```

Explanation: These lines use the 'strftime()' method to format the date and time:

- '%j': Represents the day of the year as a zero-padded number (e.g., 001, 002, ..., 365).
- '%m': Represents the month as a zero-padded number (e.g., 01, 02, ..., 12).

```
python
```

```
print('Minutes:', now.strftime('%M'))
print('AM or PM:', now.strftime('%p'))
```

Explanation: These lines use the 'strftime()' method to format the date and time:

- '%M': Represents the minute as a zero-padded number (e.g., 00, 01, ..., 59).
- '%p': Represents the AM or PM designation (e.g., AM or PM).

```
python
print('Seconds:', now.strftime('%S'))
```

Explanation: This line uses the 'strftime()' method to format the date and time:

- '%S': Represents the second as a zero-padded number (e.g., 00, 01, ..., 59).

```
python
print('Short date:', now.strftime('%x'))
print('Short time:', now.strftime('%X'))
```

Explanation: These lines use the 'strftime()' method to format the date and time:

- '%x': Represents the date using the locale's format (e.g., 07/26/2023).
- '%X': Represents the time using the locale's format (e.g., 10:45:00).

```
python
print('Short year:', now.strftime('%y'))
print('Long year:', now.strftime('%Y'))
```

Explanation: These lines use the 'strftime()' method to format the date and time:

- '%y': Represents the year without the century as a zero-padded number (e.g., 00, 01, ..., 99).
- '%Y': Represents the year with the century as a full number (e.g., 2023).

```
python
```

```
my_format = '%d/%m/%Y - %I:%M:%S %p'  
print('Date time 12:', now.strftime(my_format))
```

Explanation: These lines create a custom date and time format using a string and the 'strftime()' method:

- '%d': Day of the month as a zero-padded number.
- '%m': Month as a zero-padded number.
- '%Y': Year with century as a full number.
- '%I': Hour (12-hour clock) as a zero-padded number.
- '%M': Minute as a zero-padded number.
- '%S': Second as a zero-padded number.
- '%p': AM or PM designation.

When the code is executed, the output will show the formatted date and time values based on the current date and time at the moment of execution, similar to the format specified in the code. For example:

```
Long day name: Tuesday  
Short day name: Tue  
Long month name: July  
Short month name: Jul  
Date time: Tue Jul 26 10:45:00 2023  
Day of month: 26  
Hour number 24: 10  
Hour number 12: 10  
Day of year: 207  
Month of year: 07
```


Minutes: 45
AM or PM: AM

Seconds: 0
Short date: 07/26/2023
Short time: 10:45:00
Short year: 23
Long year: 2023
Date time 12: 26/07/2023 - 10:45:00 AM

The actual date, time, and formatting in the output will depend on the current date and time at the moment of execution.